EN3160 — Assignment 02: Fitting and Alignment

Name - Jayaweera N.S.

Index No. - 230300C

GitHub repo : https://github.com/NisalJayaweera/EN3160-Fitting-and-Alignment

This notebook combines the **Q1–Q2**, **Q3**, and **Q4** notebooks into a single ordered submission. It retains the original mathematical methods, plots, and discussions. Q3 contains both required image-pair experiments, with **45% opacity**.

```
from pathlib import Path
import os, subprocess, sys

REPO_URL = "https://github.com/NisalJayaweera/EN3160-Fitting-and-Alignment.git"
REPO_DIR = Path("/content/EN3160-Fitting-and-Alignment") if Path("/content").is_dir() else Path.cwd() / "EN3160-Fitting-and-Alignment"
if not (REPO_DIR / ".git").exists():
    subprocess.run(["git", "clone", REPO_URL, str(REPO_DIR)], check=True)
subprocess.run([sys.executable, "-m", "pip", "install", "-q", "-r", str(REPO_DIR / "requirements.txt")], check=True)
os.chdir(REPO_DIR)
print("Working directory:", Path.cwd())
```

```
Working directory: /content/EN3160-Fitting-and-Alignment
```

Question 1 — Sunflower blob detection

Objective. Detect approximately circular flower heads using scale-normalized Laplacian of Gaussian (LoG) responses and scale-space extrema rather than `cv2.HoughCircles`. The repository script performs the detection; this notebook records its execution and compares the resulting visualizations.

Environment. The shared setup cell clones the repository and installs dependencies. The following cell locates the sunflower image, with an optional upload fallback.

Q1 input preparation

The assignment sunflower photograph is loaded from the repository's `images/` directory, using a recursive search to accommodate subfolders. If absent, the notebook attempts to fetch the JPEG from the repository's GitHub `images/` folder. A Colab upload is used only as a fallback. The detection script itself reads the image with `cv2.IMREAD_REDUCED_COLOR_4`, as required. **The notebook does not use a Hough-circle detector.**

```
from pathlib import Path
from urllib.request import urlretrieve
import zipfile

SUNFLOWER_NAME = "the_berry_farms_sunflower_field.jpeg"
IMAGE_ROOT = Path.cwd() / "images"
IMAGE_ROOT.mkdir(parents=True, exist_ok=True)
found = list(IMAGE_ROOT.rglob(SUNFLOWER_NAME))
if not found:
    target = IMAGE_ROOT / SUNFLOWER_NAME
    url = "https://raw.githubusercontent.com/NisalJayaweera/EN3160-
Fitting-and-Alignment/main/images/" + SUNFLOWER_NAME
    try:
        urlretrieve(url, target)
        if target.stat().st_size > 0:
            found = [target]
    except Exception as exc:
        target.unlink(missing_ok=True)
        print("Sunflower image not found at the root GitHub image
URL:", exc)
if not found and Path("/content").is_dir():
    from google.colab import files
    print("Upload the assignment ZIP or the sunflower JPEG to continue
Q1.")
    uploaded = files.upload()
    for name, data in uploaded.items():
        if name.lower().endswith(".zip"):
            archive = IMAGE_ROOT / Path(name).name
            archive.write_bytes(data)
            with zipfile.ZipFile(archive) as z:
                z.extractall(IMAGE_ROOT)
        elif Path(name).name == SUNFLOWER_NAME:
            (IMAGE_ROOT / SUNFLOWER_NAME).write_bytes(data)
    found = list(IMAGE_ROOT.rglob(SUNFLOWER_NAME))
if not found:
    raise FileNotFoundError("Sunflower image missing. Put it in the
repository images/ folder or run this notebook in Colab and upload the
image/ZIP.")
SUNFLOWER_PATH = found[0].resolve()
print("Sunflower image:", SUNFLOWER_PATH)
```

```
Sunflower image:
/content/EN3160-Fitting-and-Alignment/images/the_berry_farms_sunflower
_field.jpeg
```

```
print("Resolved sunflower image:", SUNFLOWER_PATH)
print("Exists:", SUNFLOWER_PATH.is_file())
```

```
Resolved sunflower image:
/content/EN3160-Fitting-and-Alignment/images/the_berry_farms_sunflower
_field.jpeg
Exists: True
```

Detector implementation and parameter audit

The repository script `src/q1_blob_detection.py` performs the required scale-normalized LoG method, not a Hough-circle transform. For each scale it computes the bright-blob response $L(x, y; \sigma) = -\sigma^2 \nabla^2 (G_\sigma * I)$ and finds local maxima in a **3 × 11 × 11** neighborhood across scale and image coordinates. The negative sign targets bright sunflower centers relative to darker surroundings. Overlapping candidates are suppressed by a greedy response-ranked spatial test. The corresponding circle radius is $r = \sqrt{2}\sigma$.

Exact baseline parameters (script defaults): reduced-resolution color image (`cv2.IMREAD_REDUCED_COLOR_4`); $\sigma \in [2, 16)$ pixels with **22 linearly spaced scales**, step $14/21 \approx 0.667$ pixels; response threshold **0.025**; overlap suppression factor **0.5**; and a report of the **10 largest circles by radius**. Coordinates and radii are in the quarter-resolution image, not full-resolution source pixels. The largest permitted radius is $\sqrt{2} \times 16 \approx 22.63$ reduced-image pixels.

Important interpretation: a detected LoG extremum is a circular bright structure, not necessarily an individual sunflower. Large candidates at the maximum tested scale can be truncated by the scale boundary. The HSV stage is a post-detection color plausibility filter, not part of the LoG extrema calculation.

Baseline LoG detection

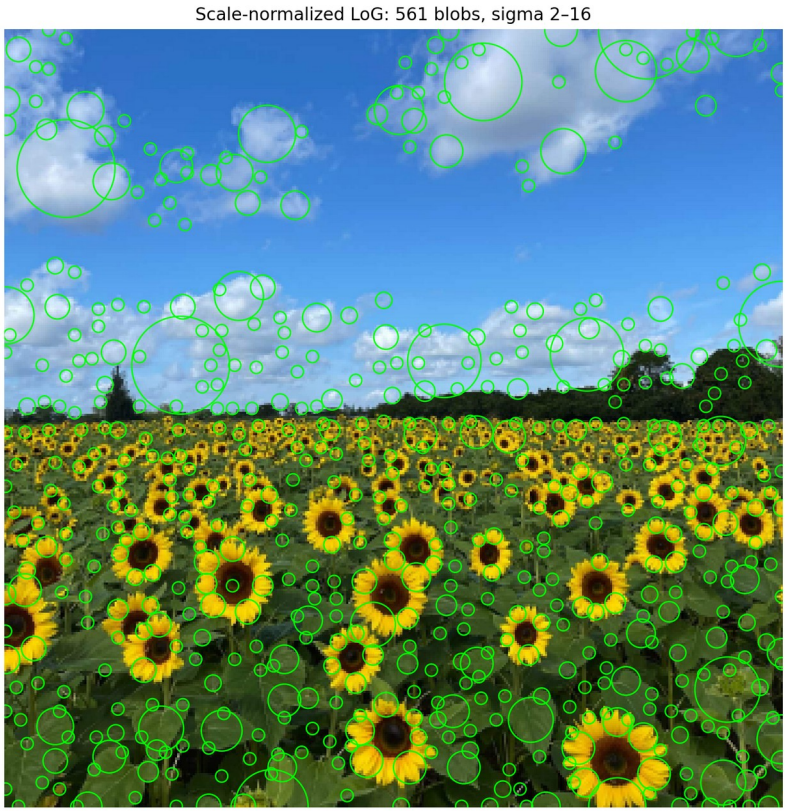
The baseline run uses `src/q1_blob_detection.py` on the extracted sunflower photograph. In scale-space blob detection, the characteristic radius of a LoG extremum is approximately (r=\sqrt{2}\sigma). The usable scale range, actual extrema selection, and reported largest circles must be checked against the script output rather than inferred from the image alone.

```
import subprocess, sys
subprocess.run([sys.executable, "src/q1_blob_detection.py", "--image",
str(SUNFLOWER_PATH)], check=True)
```

```
CompletedProcess(args=['/usr/bin/python3', 'src/q1_blob_detection.py',
'--image',
'/content/EN3160-Fitting-and-Alignment/images/the_berry_farms_sunflowe
r_field.jpeg'], returncode=0)
```

```
from IPython.display import Image, display

display(Image(
    filename="results/q1/detected_circles.png"
))
```



The repository is cloned once at the beginning of this consolidated notebook. No mid-experiment `git pull` is performed, so code does not change during execution.

HSV-filtered detection

The second run enables the script's color filter with `--yellow-fraction 0.12`. This is a useful domain-specific check because sunflower heads often contain substantial yellow regions; however, shadows, green surroundings, and partially visible flowers can make color filtering imperfect. The comparison below concerns detection selectivity, not a ground-truth accuracy measurement.

```
subprocess.run([sys.executable, "src/q1_blob_detection.py", "--image",
str(SUNFLOWER_PATH), "--color-filter", "--yellow-fraction", "0.12"],
check=True)
```

```
CompletedProcess(args=['/usr/bin/python3', 'src/q1_blob_detection.py',
'--image',
'/content/EN3160-Fitting-and-Alignment/images/the_berry_farms_sunflowe
```

```
r_field.jpeg', '--color-filter', '--yellow-fraction', '0.12'],
returncode=0)

from pathlib import Path
import pandas as pd
from PIL import Image
import matplotlib.pyplot as plt

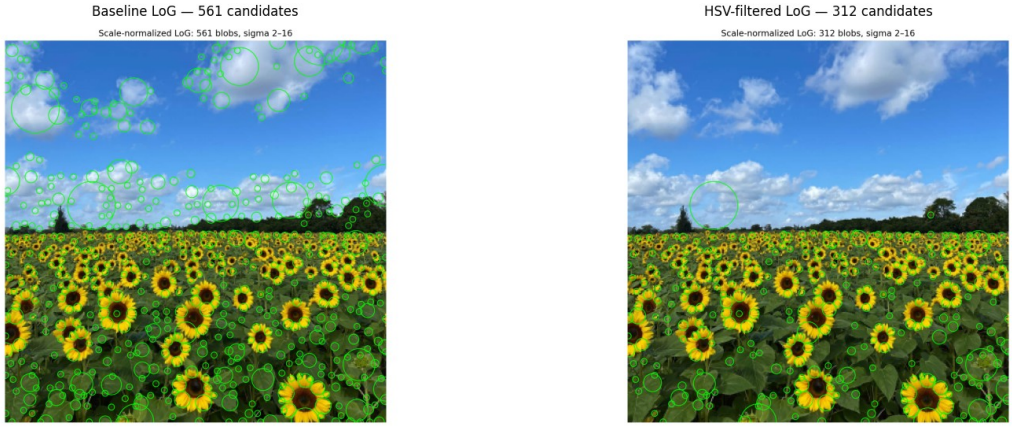
original = Image.open("results/q1/detected_circles.png")
improved = Image.open("results/q1_filtered/detected_circles.png")

fig, axes = plt.subplots(1, 2, figsize=(16, 9))

axes[0].imshow(original)
base_csv = Path("results/q1/circles.csv")
filtered_csv = Path("results/q1_filtered/circles.csv")
base_count = len(pd.read_csv(base_csv)) if base_csv.exists() else None
filtered_count = len(pd.read_csv(filtered_csv)) if
filtered_csv.exists() else None
axes[0].set_title(f"Baseline LoG — {base_count} candidates" if
base_count is not None else "Baseline LoG")
axes[0].axis("off")

axes[1].imshow(improved)
axes[1].set_title(f"HSV-filtered LoG — {filtered_count} candidates" if
filtered_count is not None else "HSV-filtered LoG")
axes[1].axis("off")

plt.tight_layout()
plt.show()
```



Q1 observations and limitations

The two panels compare the **baseline scale-normalized LoG** detector with the same detector followed by an **HSV-based yellow-content filter**. Candidate counts are read from the generated CSV files rather than assumed in advance. Fewer candidates may indicate that foliage or texture

responses were removed, but **candidate reduction is not itself a precision or recall measurement**. Manual annotations would be required to quantify accuracy.

The detector is sensitive to illumination, occlusion, overlap between flower heads, and the sampled scale range. In particular, a largest circle detected at the upper scale boundary ($\sigma = 16$ reduced-image pixels) may be scale-truncated; a wider σ sweep would be required to verify that its size is stable.

Reproducible Q1 summary and largest-circle table

The following cell reads the actual detector CSV files instead of relying on hard-coded counts. It reports the largest circles from each run and checks whether the saved output files exist. Run both detection cells first.

```
from pathlib import Path
import pandas as pd
from IPython.display import display

q1_tables = {}
for label, folder in [("Baseline LoG", "results/q1"), ("HSV-filtered
LoG", "results/q1_filtered")]:
    csv_file = Path(folder) / "circles.csv"
    if not csv_file.is_file():
        raise FileNotFoundError(f"Missing {csv_file}. Run both Q1
detector cells first.")
    table = pd.read_csv(csv_file)
    if "radius_px" not in table.columns:
        raise KeyError(f"Unexpected columns in {csv_file}:
{list(table.columns)}")
    table = table.sort_values("radius_px",
ascending=False).reset_index(drop=True)
    q1_tables[label] = table
    print(f"\n{label}: {len(table)} candidates | σ ∈ [2, 16], 22
sampled scales")
    if not table.empty:
        print("Largest detected circle (all recorded parameters):")
        display(table.head(1))
        print("Ten largest circles:")
        display(table.head(10))
    else:
        print("No circles detected.")

n_base, n_filtered = (len(q1_tables[k]) for k in ("Baseline LoG",
"HSV-filtered LoG"))
print(f"\nCandidate count: baseline={n_base}; HSV-
filtered={n_filtered}")
if n_base:
    print(f"HSV-filtered retention: {n_filtered/n_base:.1%}")
print("These counts are detector candidates, not verified sunflower
ground truth.")
```

```
Baseline LoG: 561 candidates | σ ∈ [2, 16], 22 sampled scales
Largest detected circle (all recorded parameters):
```


```
{"summary":"{\n  \"name\": \"print(\\\"These counts are detector
candidates, not verified sunflower ground truth\",\n  \"rows\": 1,\n
\"fields\": [\n    {\n      \"column\": \"center_x\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 297.0,\n        \"max\": 297.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          297.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"center_y\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 0.0,\n        \"max\": 0.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"sigma\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": null,\n
\"min\": 16.0,\n        \"max\": 16.0,\n        \"num_unique_values\":
1,\n        \"samples\": [\n          16.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"radius_px\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 22.627,\n        \"max\": 22.627,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          22.627\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"log_response\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 0.063766,\n        \"max\": 0.063766,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.063766\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


```
Ten largest circles:
```


```
{"summary":"{\n  \"name\": \"print(\\\"These counts are detector
candidates, not verified sunflower ground truth\",\n  \"rows\": 10,\n
\"fields\": [\n    {\n      \"column\": \"center_x\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
111.06534613060508,\n        \"min\": 28.0,\n        \"max\": 359.0,\n
\"num_unique_values\": 10,\n        \"samples\": [\n          334.0,\n
28.0,\n          203.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"center_y\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 119.74347581392483,\n        \"min\":
0.0,\n        \"max\": 359.0,\n        \"num_unique_values\": 10,\n
\"samples\": [\n          305.0,\n          64.0,\n          153.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"sigma\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
```

```
2.245385945543537,\n        \"min\": 10.0,\n        \"max\": 16.0,\n
\"num_unique_values\": 6,\n        \"samples\": [\n          16.0,\n
14.0,\n          10.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"radius_px\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 3.1752959634584537,\n        \"min\":
14.142,\n        \"max\": 22.627,\n        \"num_unique_values\": 6,\n
\"samples\": [\n          22.627,\n          19.799,\n
14.142\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"log_response\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.04231455285267968,\n        \"min\":
0.063766,\n        \"max\": 0.182128,\n        \"num_unique_values\":
10,\n        \"samples\": [\n          0.089327,\n          0.146176,\
n          0.13547\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


HSV-filtered LoG: 312 candidates | σ ∈ [2, 16], 22 sampled scales
Largest detected circle (all recorded parameters):


```
{"summary":"{\n  \"name\": \"print(\\\"These counts are detector
candidates, not verified sunflower ground truth\",\n  \"rows\": 1,\n
\"fields\": [\n    {\n      \"column\": \"center_x\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 81.0,\n        \"max\": 81.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          81.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"center_y\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 155.0,\n        \"max\": 155.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          155.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"sigma\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": null,\n
\"min\": 16.0,\n        \"max\": 16.0,\n        \"num_unique_values\":
1,\n        \"samples\": [\n          16.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"radius_px\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 22.627,\n        \"max\": 22.627,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          22.627\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"log_response\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 0.099091,\n        \"max\": 0.099091,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.099091\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```

```
Ten largest circles:

{"summary":"{\n  \"name\": \"print(\\\"These counts are detector
candidates, not verified sunflower ground truth\",\n  \"rows\": 10,\n
\"fields\": [\n    {\n      \"column\": \"center_x\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
130.06306162781192,\n        \"min\": 23.0,\n        \"max\": 359.0,\n
\"num_unique_values\": 10,\n        \"samples\": [\n          332.0,\n
123.0,\n          359.0\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"center_y\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 70.48120316793691,\n        \"min\":
155.0,\n        \"max\": 359.0,\n        \"num_unique_values\": 9,\n
\"samples\": [\n          348.0,\n          359.0,\n          275.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"sigma\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
2.9781956674022165,\n        \"min\": 6.0,\n        \"max\": 16.0,\n
\"num_unique_values\": 7,\n        \"samples\": [\n          16.0,\n
12.0,\n          7.333\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"radius_px\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 4.211610533856246,\n        \"min\":
8.485,\n        \"max\": 22.627,\n        \"num_unique_values\": 7,\n
\"samples\": [\n          22.627,\n          16.971,\n
10.371\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"log_response\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.03999000192395316,\n        \"min\":
0.061636,\n        \"max\": 0.180504,\n        \"num_unique_values\":
10,\n        \"samples\": [\n          0.090722,\n          0.121264,\
n          0.061636\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}


Candidate count: baseline=561; HSV-filtered=312
HSV-filtered retention: 55.6%
These counts are detector candidates, not verified sunflower ground
truth.
```

Q1 interpretation for the report

The baseline LoG detector is designed to identify scale-space extrema over multiple blob sizes. The additional HSV check removes candidates whose surrounding neighborhood contains insufficient yellow pixels, making the results more visually specific to sunflower-like regions. However, yellow color does not establish that a candidate is a flower head, and shadows, occlusion, flower orientation, and overlapping flowers can still cause false detections or missed detections. Compare the saved overlays before claiming an improvement in accuracy.

The maximum detected radius should be interpreted in relation to the **upper scale bound**, since an extremum at $\sigma = 16$ cannot be tested against larger scales. A sensitivity study using a wider scale range and different LoG response thresholds would establish whether the largest structures are stable. The assignment asks for the largest-circle parameters, which are provided by the CSV-backed table above; these are empirical detector outputs rather than known ground truth.

Question 2: Line and Circle Fitting Using RANSAC

This experiment uses RANSAC to estimate a line and a circle from a noisy point set containing observations from both models.

The line is estimated first using perpendicular-distance errors. Its consensus points are then removed, and a circle is fitted to the remaining points using radial-distance errors.

The RANSAC algorithms are implemented from scratch, and the final models are refined using their respective consensus sets.

Question 2 — Sequential RANSAC line and circle estimation

Dataset and reproducibility. The assignment's synthetic dataset contains 100 points: 50 sampled around a circle with true center ((2,3)), radius (10), and radial noise standard deviation (10/16=0.625); and 50 sampled near (y=-x+2), with Gaussian vertical noise standard deviation (1). A fixed NumPy seed makes the numerical experiment repeatable. Both geometric structures are present in the combined point cloud.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# Reproducibility
np.random.seed(0)

# Generate noisy circle points
N = 100
half_n = N // 2

r_gt = 10
x0_gt, y0_gt = 2, 3

s_circle = r_gt / 16

t = np.random.uniform(0, 2 * np.pi, half_n)
n = s_circle * np.random.randn(half_n)
```

```
x = x0_gt + (r_gt + n) * np.cos(t)
y = y0_gt + (r_gt + n) * np.sin(t)

X_circ = np.column_stack((x, y))

# Generate noisy line points
m_gt, b_gt = -1, 2
s_line = 1.0

x = np.linspace(-12, 12, half_n)
y = m_gt * x + b_gt + s_line * np.random.randn(half_n)

X_line = np.column_stack((x, y))

# Combined dataset
X = np.vstack((X_circ, X_line))

# Plot
fig, ax = plt.subplots(figsize=(8, 8))

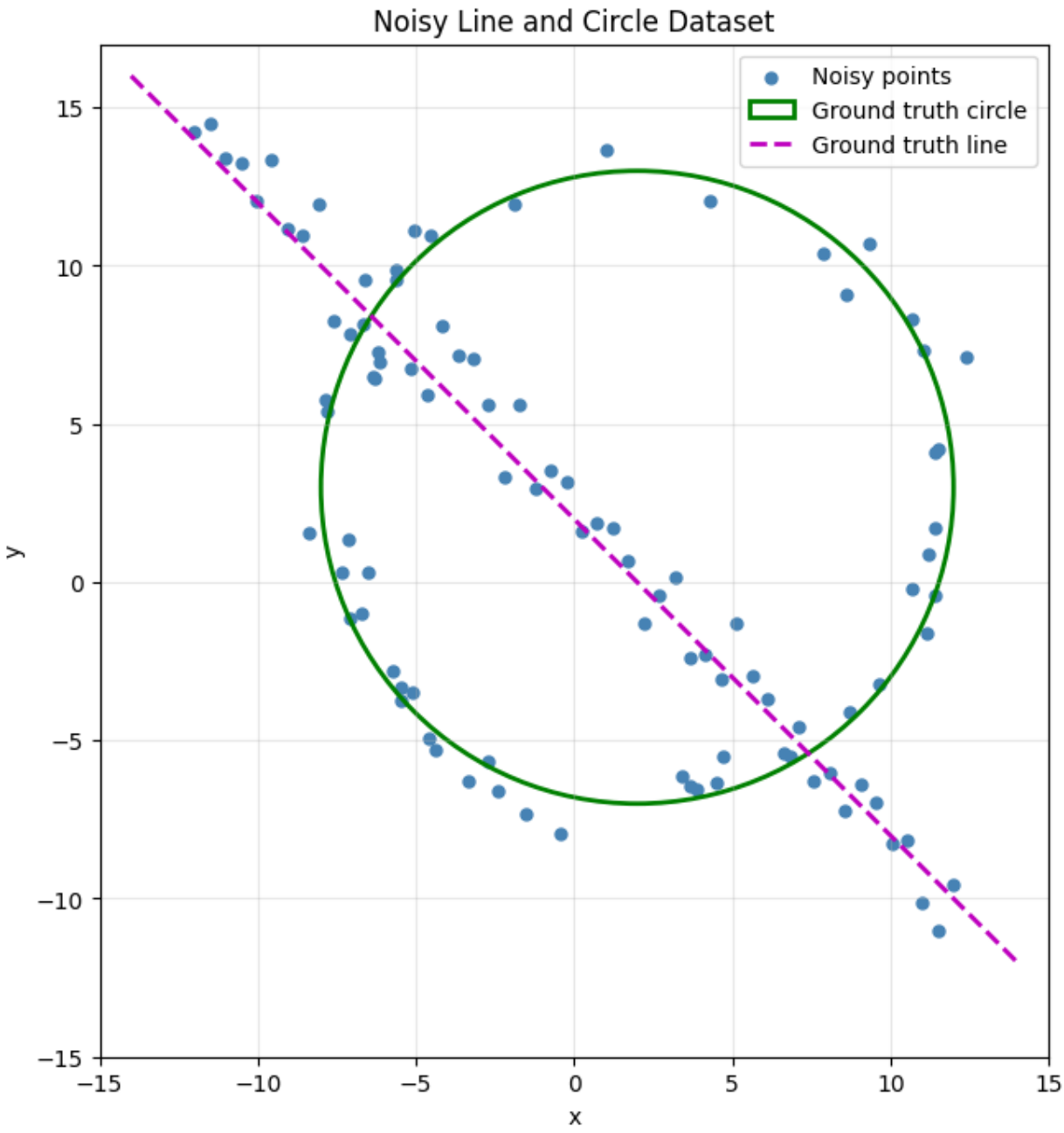
ax.scatter(X[:, 0], X[:, 1],
           color="steelblue", s=25, label="Noisy points")

circle_gt = plt.Circle(
    (x0_gt, y0_gt), r_gt,
    fill=False, color="green",
    linewidth=2, label="Ground truth circle"
)
ax.add_patch(circle_gt)

x_range = np.linspace(-14, 14, 300)
ax.plot(x_range, m_gt * x_range + b_gt,
        "m--", linewidth=2, label="Ground truth line")

ax.set_aspect("equal")
ax.set_xlim(-15, 15)
ax.set_ylim(-15, 17)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Noisy Line and Circle Dataset")
ax.grid(alpha=0.3)
ax.legend()
plt.show()

print("Total points:", len(X))
print("Circle points:", len(X_circ))
print("Line points:", len(X_line))
```




```
Total points: 100
Circle points: 50
Line points: 50
```

2(a) — Robust line estimation

Write the line as (ax+by=d) with (a^2+b^2=1). This normalization makes (|ax_i+by_i-d|) the **perpendicular distance** of a point to the line. Each RANSAC trial draws two distinct observations, constructs a candidate normalized line, and scores its consensus using a distance

threshold of **0.8**. The highest consensus wins, with squared inlier error breaking ties. A minimum of **30** inliers is required; **3000** hypotheses are tried.

The winning two-point hypothesis is then refined by total least squares (TLS) using SVD of the consensus points. The final TLS line and the original winning sample line are deliberately shown separately: refinement reduces dependence on two noisy sample points.

The threshold is an experimental setting, not an analytically optimized one. In particular, the line noise was added vertically, not perpendicular to the line, so the effective perpendicular noise scale differs.

```
def line_from_two_points(p1, p2):
    direction = p2 - p1
    normal = np.array([-direction[1], direction[0]])
    norm = np.linalg.norm(normal)

    if norm < 1e-12:
        return None

    normal = normal / norm
    a, b = normal
    d = np.dot(normal, p1)

    return np.array([a, b, d])


def line_errors(points, model):
    a, b, d = model
    return np.abs(points[:, 0] * a +
                  points[:, 1] * b - d)


def fit_line_tls(points):
    centroid = np.mean(points, axis=0)
    centered = points - centroid

    _, _, Vt = np.linalg.svd(centered)
    normal = Vt[-1]
    normal = normal / np.linalg.norm(normal)

    d = np.dot(normal, centroid)

    return np.array([normal[0], normal[1], d])


def ransac_line(points, threshold=0.8,
                iterations=3000, min_inliers=30, seed=42):

    rng = np.random.default_rng(seed)
```

```
    best_model = None
    best_sample_model = None
    best_sample_indices = None
    best_inliers = None
    best_count = 0
    best_error = np.inf

    for _ in range(iterations):
        indices = rng.choice(len(points), 2, replace=False)
        model = line_from_two_points(*points[indices])

        if model is None:
            continue

        errors = line_errors(points, model)
        inliers = errors < threshold

        count = np.sum(inliers)
        total_error = np.sum(errors[inliers] ** 2)

        if (count > best_count or
            (count == best_count and total_error < best_error)):

            best_count = count
            best_error = total_error
            best_sample_model = model
            best_sample_indices = indices
            best_inliers = inliers

    if best_count < min_inliers:
        raise RuntimeError("Insufficient line consensus")

    best_model = fit_line_tls(points[best_inliers])

    return {
        "model": best_model,
        "sample_model": best_sample_model,
        "sample_indices": best_sample_indices,
        "inliers": best_inliers,
        "count": best_count
    }


line_result = ransac_line(X)

a, b, d = line_result["model"]

print("Estimated line parameters:")
print(f"a = {a:.4f}")
print(f"b = {b:.4f}")
```

```
print(f"d = {d:.4f}")
print(f"Normal magnitude = {np.hypot(a, b):.6f}")
print(f"Number of line inliers = {line_result['count']}")
```

```
Estimated line parameters:
a = 0.7124
b = 0.7018
d = 1.4255
Normal magnitude = 1.000000
Number of line inliers = 41
```

```
def plot_line(ax, model, x_values, **kwargs):
    a, b, d = model

    if abs(b) > 1e-10:
        y_values = (d - a * x_values) / b
        ax.plot(x_values, y_values, **kwargs)
    else:
        ax.axvline(d / a, **kwargs)


fig, ax = plt.subplots(figsize=(8, 8))

ax.scatter(X[:, 0], X[:, 1],
           s=20, color="lightgray", label="All points")

ax.scatter(
    X[line_result["inliers"], 0],
    X[line_result["inliers"], 1],
    s=35, color="orange", label="Line inliers"
)

sample = X[line_result["sample_indices"]]
ax.scatter(sample[:, 0], sample[:, 1],
           color="red", marker="x",
           s=120, linewidth=2, label="Best line sample")

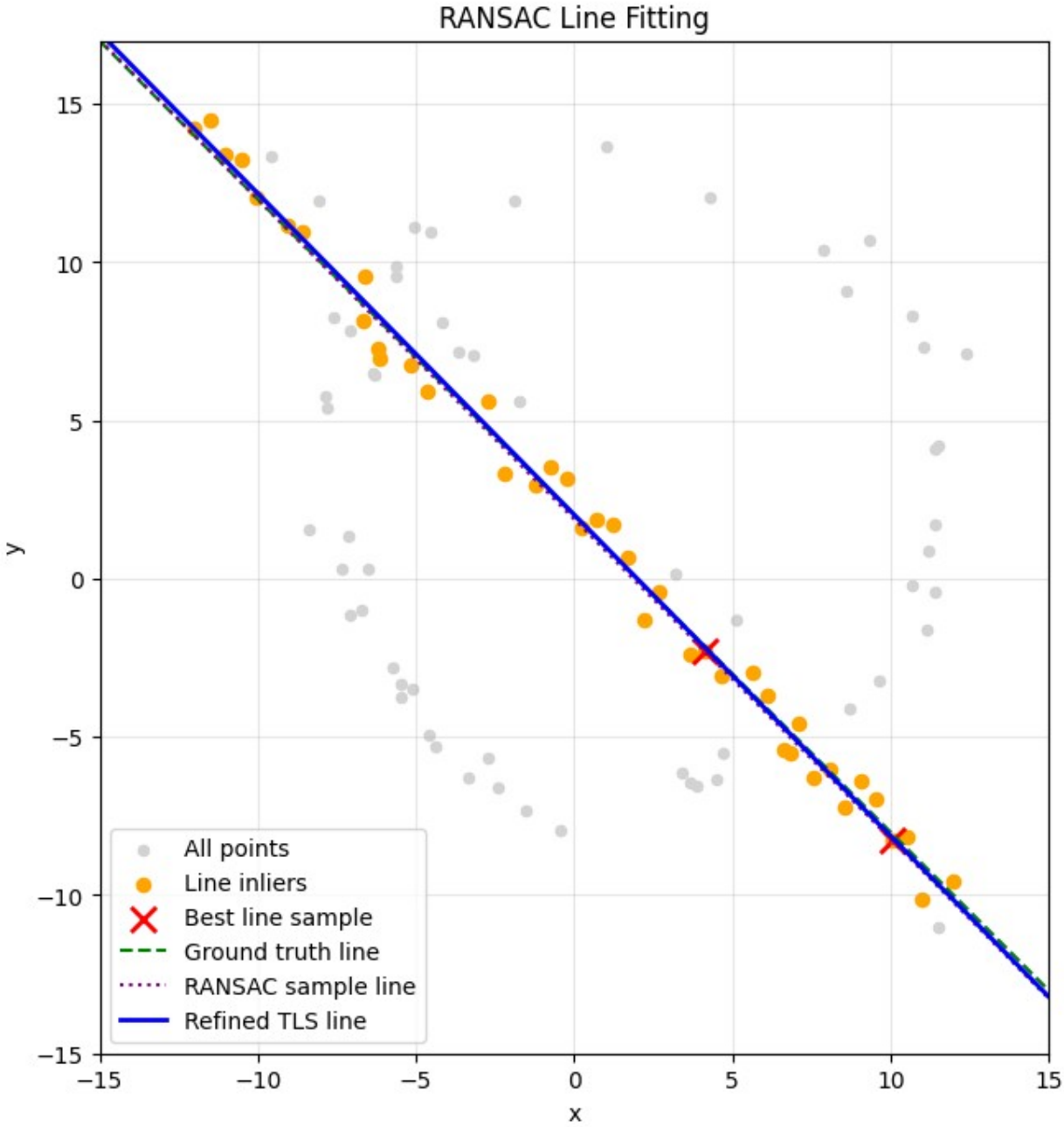
x_values = np.linspace(-15, 15, 300)

ax.plot(x_values, m_gt * x_values + b_gt,
        "g--", label="Ground truth line")

plot_line(ax, line_result["sample_model"], x_values,
          color="purple", linestyle=":",
          label="RANSAC sample line")

plot_line(ax, line_result["model"], x_values,
          color="blue", linewidth=2,
          label="Refined TLS line")
```

```
ax.set_xlim(-15, 15)
ax.set_ylim(-15, 17)
ax.set_aspect("equal")
ax.set_title("RANSAC Line Fitting")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.grid(alpha=0.3)
ax.legend()
plt.show()
```



Line-inlier interpretation

Points highlighted as line inliers lie within the chosen perpendicular-distance threshold of the **winning sample hypothesis**. The refined TLS line is fitted to these points. Because the algorithm does not recompute consensus after TLS refinement, some points might cross the threshold relative to the final line. Points from the circle that happen to lie near the line can also be assigned to its consensus.

```
# Remove the line consensus points
line_inlier_mask = line_result["inliers"]

X_remaining = X[~line_inlier_mask]

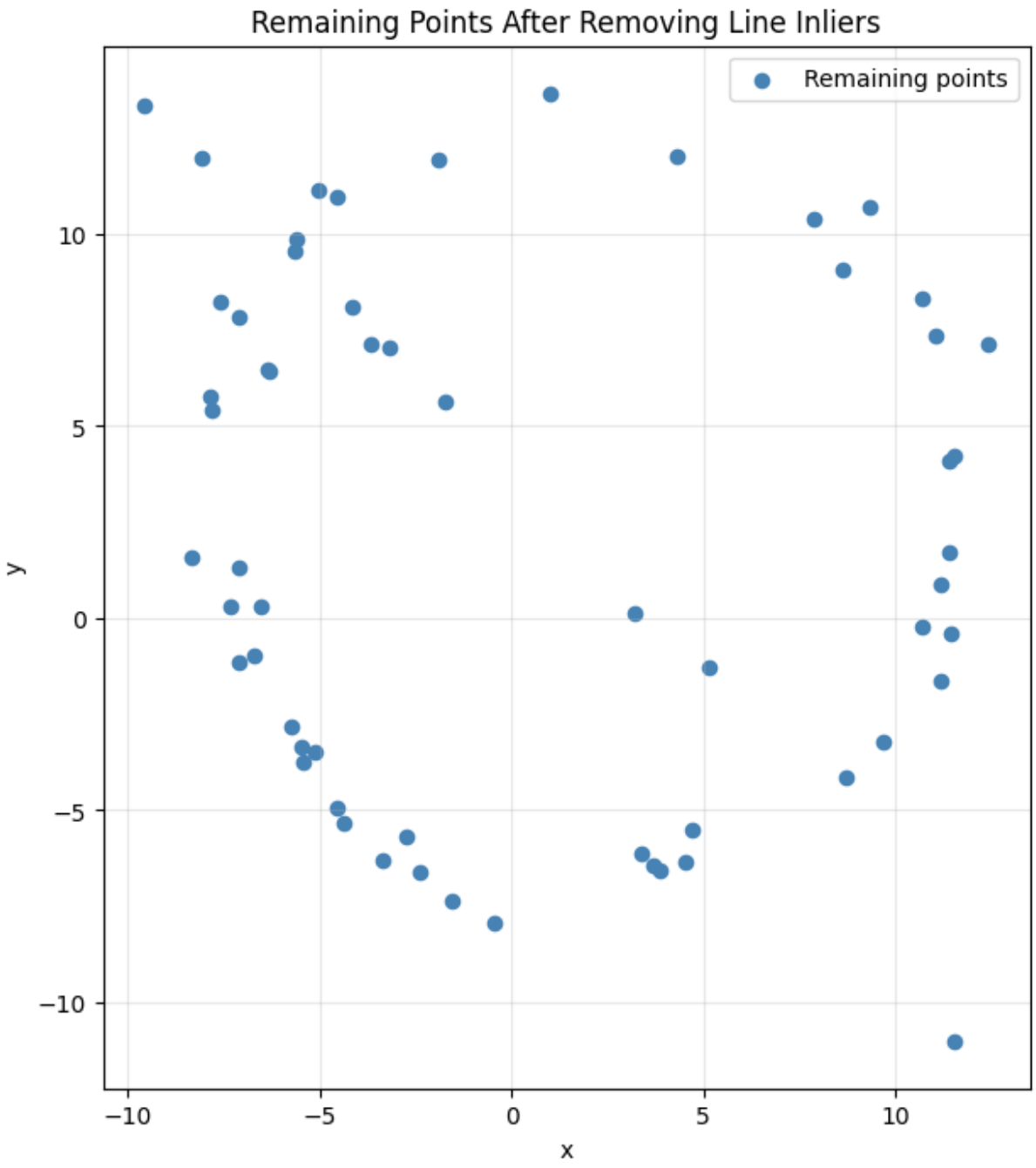
print("Total original points:", len(X))
print("Line inliers removed:", np.sum(line_inlier_mask))
print("Remaining points:", len(X_remaining))

# Visualize remaining points
plt.figure(figsize=(8, 8))

plt.scatter(
    X_remaining[:, 0],
    X_remaining[:, 1],
    color="steelblue",
    s=35,
    label="Remaining points"
)

plt.gca().set_aspect("equal")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Remaining Points After Removing Line Inliers")
plt.grid(alpha=0.3)
plt.legend()
plt.show()
```

```
Total original points: 100
Line inliers removed: 41
Remaining points: 59
```



2(b) — Remove line consensus, then fit the circle

RANSAC is applied to `X_remaining`, formed by excluding the selected line inliers. Each circle hypothesis uses **three non-collinear points**, which uniquely determine a center and radius. Candidate circles are evaluated by **radial residual** (e_i=\left|\sqrt{(x_i-x_c)^2+(y_i-y_c)^2}-r\ right|). The implementation uses threshold **1.0**, **5000** iterations, and minimum consensus **25**.

The radial-noise standard deviation in the simulated circle is 0.625, making 1.0 a plausible exploratory threshold. The selected circle is refined by minimizing the sum of squared radial

residuals over its consensus, with a positive-radius bound. As with the line, the final model is refined without recomputing the inlier mask.

```
def circle_from_three_points(p1, p2, p3):
    """
    Estimate a circle passing through three points.
    Returns (xc, yc, r), or None for collinear points.
    """
    A = 2 * np.array([
        p2 - p1,
        p3 - p1
    ])

    B = np.array([
        np.dot(p2, p2) - np.dot(p1, p1),
        np.dot(p3, p3) - np.dot(p1, p1)
    ])

    if abs(np.linalg.det(A)) < 1e-8:
        return None

    center = np.linalg.solve(A, B)
    radius = np.linalg.norm(p1 - center)

    if not np.isfinite(radius):
        return None

    return np.array([center[0], center[1], radius])


def circle_errors(points, model):
    xc, yc, r = model

    distances = np.sqrt(
        (points[:, 0] - xc)**2 +
        (points[:, 1] - yc)**2
    )

    return np.abs(distances - r)


def fit_circle_least_squares(points, initial_model):
    """
    Refine the circle by minimizing squared radial errors.
    """
    def objective(params):
        errors = circle_errors(points, params)
        return np.sum(errors**2)

    result = minimize(
```

```
        objective,
        initial_model,
        method="L-BFGS-B",
        bounds=[
            (None, None),
            (None, None),
            (1e-6, None)
        ]
    )

    if not result.success:
        raise RuntimeError(
            f"Circle optimization failed: {result.message}"
        )

    return result.x


def ransac_circle(points, threshold=1.0,
                  iterations=5000,
                  min_inliers=25,
                  seed=42):

    rng = np.random.default_rng(seed)

    best_sample_model = None
    best_sample_indices = None
    best_inliers = None
    best_count = 0
    best_error = np.inf

    for _ in range(iterations):

        indices = rng.choice(
            len(points), 3, replace=False
        )

        sample = points[indices]

        model = circle_from_three_points(
            sample[0], sample[1], sample[2]
        )

        if model is None:
            continue

        errors = circle_errors(points, model)
        inliers = errors < threshold

        count = np.sum(inliers)
```

```
        total_error = np.sum(errors[inliers]**2)

        if (count > best_count or
            (count == best_count and
             total_error < best_error)):

            best_count = count
            best_error = total_error
            best_sample_model = model
            best_sample_indices = indices
            best_inliers = inliers.copy()

    if best_count < min_inliers:
        raise RuntimeError(
            "Insufficient circle consensus"
        )

    refined_model = fit_circle_least_squares(
        points[best_inliers],
        best_sample_model
    )

    return {
        "model": refined_model,
        "sample_model": best_sample_model,
        "sample_indices": best_sample_indices,
        "inliers": best_inliers,
        "count": best_count
    }


# Run circle RANSAC on remaining points
circle_result = ransac_circle(X_remaining)

xc, yc, r = circle_result["model"]

print("Estimated circle parameters:")
print(f"Center x = {xc:.4f}")
print(f"Center y = {yc:.4f}")
print(f"Radius   = {r:.4f}")
print(f"Circle inliers = {circle_result['count']}")
```

```
Estimated circle parameters:
Center x = 1.9751
Center y = 3.0746
Radius   = 9.9363
Circle inliers = 46
```

```
fig, ax = plt.subplots(figsize=(8, 8))
```

```
# Remaining points
ax.scatter(
    X_remaining[:, 0],
    X_remaining[:, 1],
    s=25,
    color="lightgray",
    label="Remaining points"
)

# Circle inliers
circle_inliers = X_remaining[circle_result["inliers"]]

ax.scatter(
    circle_inliers[:, 0],
    circle_inliers[:, 1],
    s=35,
    color="cyan",
    label="Circle inliers"
)

# Best RANSAC sample
sample = X_remaining[circle_result["sample_indices"]]

ax.scatter(
    sample[:, 0],
    sample[:, 1],
    s=100,
    color="red",
    marker="x",
    linewidth=2,
    label="Best circle sample (3 points)"
)

# Ground truth circle
gt_circle = plt.Circle(
    (x0_gt, y0_gt),
    r_gt,
    fill=False,
    edgecolor="green",
    linewidth=2,
    linestyle="--",
    label="Ground truth circle"
)
ax.add_patch(gt_circle)

# Circle from best RANSAC sample
sx, sy, sr = circle_result["sample_model"]

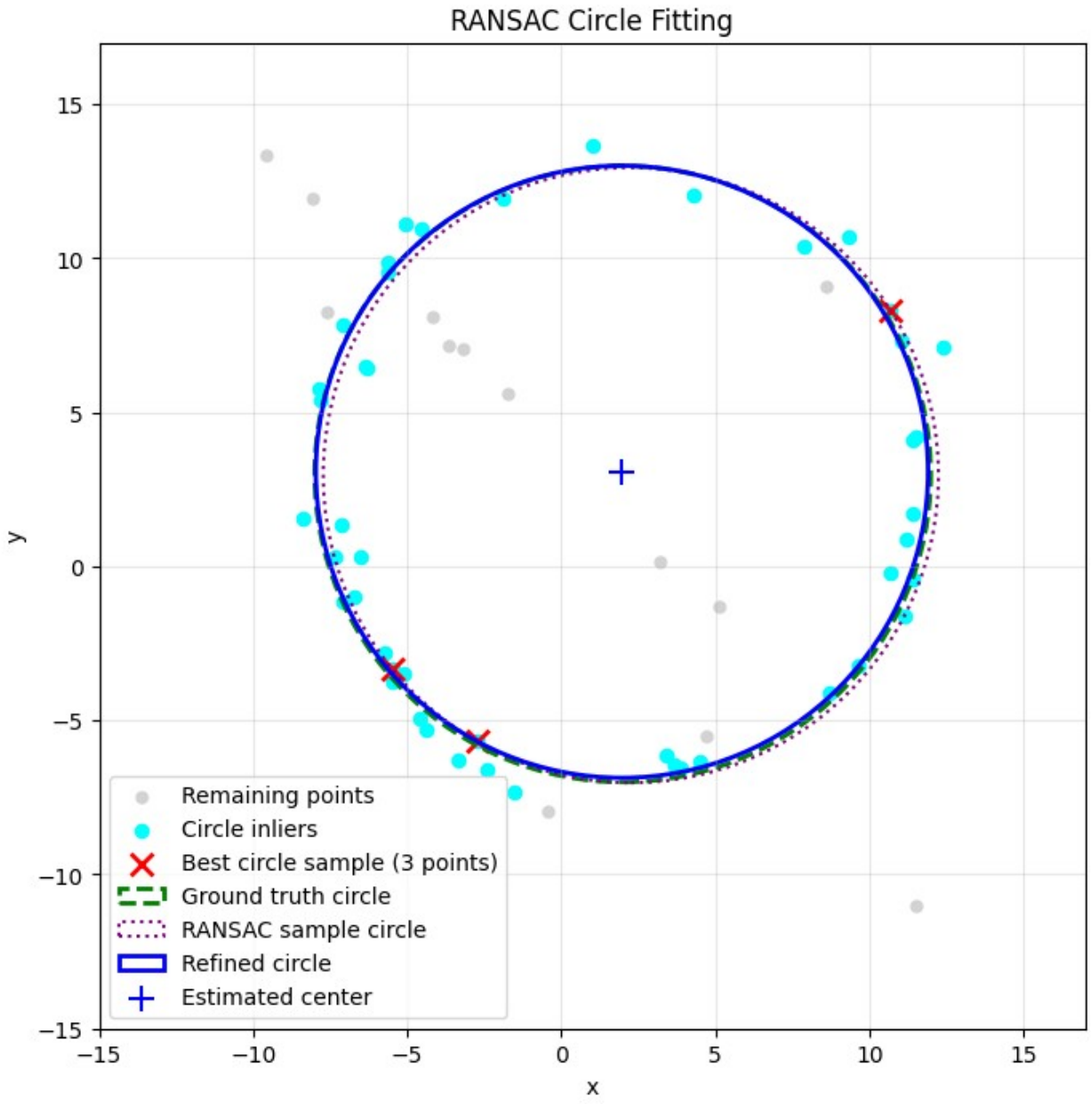
sample_circle = plt.Circle(
    (sx, sy),
```

```
    sr,
    fill=False,
    edgecolor="purple",
    linewidth=1.5,
    linestyle=":",
    label="RANSAC sample circle"
)
ax.add_patch(sample_circle)

# Refined circle
fitted_circle = plt.Circle(
    (xc, yc),
    r,
    fill=False,
    edgecolor="blue",
    linewidth=2,
    label="Refined circle"
)
ax.add_patch(fitted_circle)

ax.scatter(
    [xc], [yc],
    marker="+",
    s=120,
    color="blue",
    label="Estimated center"
)

ax.set_xlim(-15, 17)
ax.set_ylim(-15, 17)
ax.set_aspect("equal")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("RANSAC Circle Fitting")
ax.grid(alpha=0.3)
ax.legend()
plt.show()
```




```
print("Circle Parameter Comparison")
print("-" * 48)
print(f"{'Parameter':<15}{'Ground Truth':<15}{'Estimated':<15}")
print("-" * 48)

print(f"{'Center X':<15}{x0_gt:<15.4f}{xc:<15.4f}")
print(f"{'Center Y':<15}{y0_gt:<15.4f}{yc:<15.4f}")
print(f"{'Radius':<15}{r_gt:<15.4f}{r:<15.4f}")

center_error = np.linalg.norm(
    np.array([xc, yc]) - np.array([x0_gt, y0_gt])
```

```
)

radius_error = abs(r - r_gt)

print("\nEstimation Errors")
print(f"Center error: {center_error:.4f}")
print(f"Radius error: {radius_error:.4f}")

# Radial RMSE on the RANSAC consensus
inlier_errors = circle_errors(
    circle_inliers, circle_result["model"]
)

rmse = np.sqrt(np.mean(inlier_errors**2))

print(f"Circle inlier RMSE: {rmse:.4f}")
```

```
Circle Parameter Comparison
------------------------------------------------
Parameter      Ground Truth   Estimated
------------------------------------------------
Center X       2.0000         1.9751
Center Y       3.0000         3.0746
Radius         10.0000        9.9363

Estimation Errors
Center error: 0.0787
Radius error: 0.0637
Circle inlier RMSE: 0.5281
```

Circle results and uncertainty

Compare the estimated center with ((2,3)), the radius with (10), and the sample hypothesis with the refined circle. The circle-inlier radial RMSE measures fit quality **on the selected consensus only**; it is not a generalization error and may be optimistic because those points were selected using a threshold. Some original circle observations may have been removed during the preceding line fit.

```
# ============================================
# Task 2(c): Combined RANSAC Visualization
# ============================================

fig, ax = plt.subplots(figsize=(10, 10))

# 1. Original noisy points
ax.scatter(
    X[:, 0], X[:, 1],
    s=25, color="gray", alpha=0.45,
    label="All points", zorder=1
```

```
)

# 2. Line inliers
line_inliers = X[line_result["inliers"]]

ax.scatter(
    line_inliers[:, 0], line_inliers[:, 1],
    s=35, color="gold",
    label="Line inliers", zorder=2
)

# 3. Circle inliers
circle_inliers = X_remaining[circle_result["inliers"]]

ax.scatter(
    circle_inliers[:, 0], circle_inliers[:, 1],
    s=35, color="darkturquoise",
    label="Circle inliers", zorder=3
)

# 4. Best RANSAC sample for line
line_sample = X[line_result["sample_indices"]]

ax.scatter(
    line_sample[:, 0], line_sample[:, 1],
    s=120, color="red", marker="x",
    linewidth=2.5,
    label="Best sample for line", zorder=6
)

# 5. Best RANSAC sample for circle
circle_sample = X_remaining[circle_result["sample_indices"]]

ax.scatter(
    circle_sample[:, 0], circle_sample[:, 1],
    s=100, color="darkred", marker="o",
    edgecolor="black",
    label="Best sample for circle", zorder=6
)

# 6. Ground truth line
x_values = np.linspace(-15, 15, 400)

ax.plot(
    x_values,
    m_gt * x_values + b_gt,
    color="green", linestyle="--",
    linewidth=1.8,
    label="Ground truth line"
)
```

```python
# 7. Line from best RANSAC sample
plot_line(
    ax,
    line_result["sample_model"],
    x_values,
    color="purple", linestyle=":",
    linewidth=1.8,
    label="RANSAC sample line"
)

# 8. Refined line using TLS
plot_line(
    ax,
    line_result["model"],
    x_values,
    color="blue",
    linewidth=2.2,
    label="Refined RANSAC line"
)

# 9. Ground truth circle
ground_truth_circle = plt.Circle(
    (x0_gt, y0_gt),
    r_gt,
    fill=False,
    edgecolor="green",
    linestyle="--",
    linewidth=1.8,
    label="Ground truth circle"
)
ax.add_patch(ground_truth_circle)

# 10. Circle from best RANSAC sample
sx, sy, sr = circle_result["sample_model"]

sample_circle = plt.Circle(
    (sx, sy),
    sr,
    fill=False,
    edgecolor="purple",
    linestyle=":",
    linewidth=1.8,
    label="RANSAC sample circle"
)
ax.add_patch(sample_circle)

# 11. Refined circle
xc, yc, r = circle_result["model"]
```

```
refined_circle = plt.Circle(
    (xc, yc),
    r,
    fill=False,
    edgecolor="blue",
    linewidth=2.2,
    label="Refined RANSAC circle"
)
ax.add_patch(refined_circle)

# 12. Estimated circle center
ax.scatter(
    xc, yc,
    marker="+", s=180,
    linewidth=2.5,
    color="blue",
    label="Estimated circle center",
    zorder=7
)

# Figure formatting
ax.set_xlim(-15, 17)
ax.set_ylim(-15, 17)
ax.set_aspect("equal", adjustable="box")

ax.set_xlabel("X coordinate", fontsize=12)
ax.set_ylabel("Y coordinate", fontsize=12)

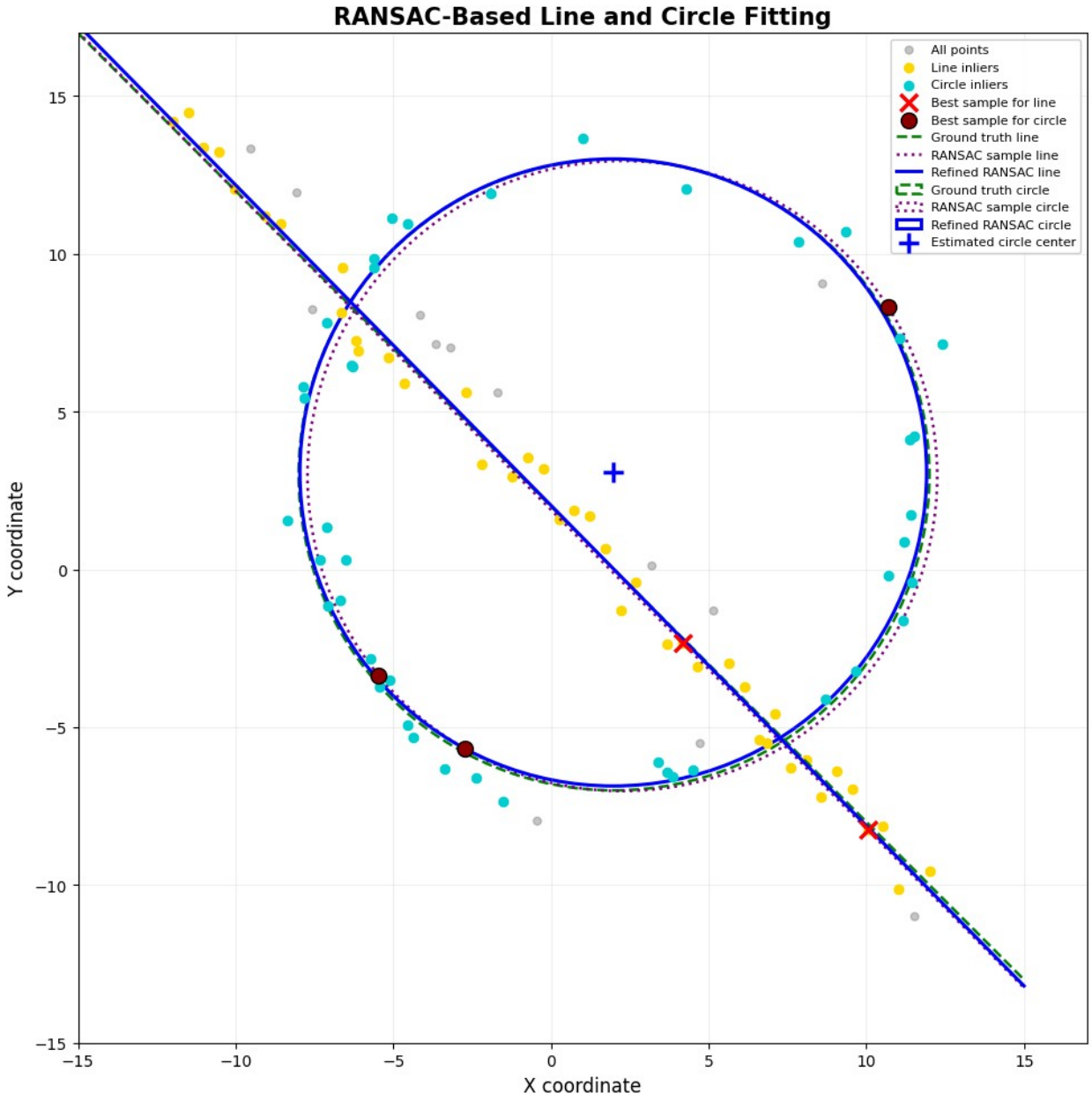
ax.set_title(
    "RANSAC-Based Line and Circle Fitting",
    fontsize=15, fontweight="bold"
)

ax.grid(True, alpha=0.2)
ax.legend(loc="upper right", fontsize=8, framealpha=0.95)

plt.tight_layout()

# Save for LaTeX report
plt.savefig(
    "Q2_RANSAC_Combined_Result.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



2(c) — Joint visualization

The combined figure shows the full cloud, both sequentially assigned consensus sets, the winning 2-point line sample and 3-point circle sample, the corresponding sample hypotheses, the refined fits, and both true models. Separation of sample and refined models demonstrates why RANSAC consensus selection is followed by least-squares fitting. The line and circle consensus groups are disjoint **by construction**, not because the geometric models cannot overlap.

```
# ============================================
# Quantitative Evaluation
```

```python
# ============================================

# Ground truth normalized line
gt_normal = np.array([-m_gt, 1.0])
gt_normal /= np.linalg.norm(gt_normal)
gt_d = b_gt / np.sqrt(m_gt**2 + 1)

# Estimated line
a_est, b_est, d_est = line_result["model"]

# Account for sign ambiguity
estimated_normal = np.array([a_est, b_est])

if np.dot(estimated_normal, gt_normal) < 0:
    estimated_normal = -estimated_normal
    d_est = -d_est

# Angular error between normals
cos_angle = np.clip(
    np.dot(estimated_normal, gt_normal), -1, 1
)
angular_error = np.degrees(np.arccos(cos_angle))

# Perpendicular distance parameter error
distance_error = abs(d_est - gt_d)

# Circle parameter errors
xc, yc, r = circle_result["model"]

center_error = np.hypot(xc - x0_gt, yc - y0_gt)
radius_error = abs(r - r_gt)

print("=" * 50)
print("RANSAC MODEL EVALUATION")
print("=" * 50)

print("\nLINE FITTING")
print(f"Line inliers       : {line_result['count']}")
print(f"Angular error      : {angular_error:.4f} degrees")
print(f"Distance error     : {distance_error:.4f}")

print("\nCIRCLE FITTING")
print(f"Circle inliers     : {circle_result['count']}")
print(f"Center error       : {center_error:.4f}")
print(f"Radius error       : {radius_error:.4f}")

print("\nMODEL PARAMETERS")
print(f"Estimated line     : {a_est:.4f}x + "
      f"{b_est:.4f}y = {line_result['model'][2]:.4f}")
```

```
print(f"Estimated circle   : center=({xc:.4f}, {yc:.4f}), "
      f"radius={r:.4f}")

==================================================
RANSAC MODEL EVALUATION
==================================================

LINE FITTING
Line inliers       : 41
Angular error      : 0.4318 degrees
Distance error     : 0.0113

CIRCLE FITTING
Circle inliers     : 46
Center error       : 0.0787
Radius error       : 0.0637

MODEL PARAMETERS
Estimated line     : 0.7124x + 0.7018y = 1.4255
Estimated circle   : center=(1.9751, 3.0746), radius=9.9363
```

Quantitative evaluation

The line normal is sign-ambiguous: ((a,b,d)) and ((-a,-b,-d)) represent the same line. The evaluation aligns normals before reporting angular error and the error in (d). The circle is compared using center-position error and absolute radius error. Because the synthetic ground truth is known, these are meaningful parameter errors, in contrast to real images where ground-truth geometry is typically unavailable.

```
# ============================================
# Task 2(d): Circle-First RANSAC
# ============================================

# Step 1: Fit circle to the entire dataset
circle_first_result = ransac_circle(
    X,
    threshold=1.0,
    iterations=5000,
    min_inliers=25,
    seed=42
)

# Step 2: Remove circle consensus
circle_first_mask = circle_first_result["inliers"]
X_after_circle = X[~circle_first_mask]

# Step 3: Fit line to remaining points
line_second_result = ransac_line(
    X_after_circle,
```

```
    threshold=0.8,
    iterations=3000,
    min_inliers=30,
    seed=42
)

# Extract parameters
xc_cf, yc_cf, r_cf = circle_first_result["model"]
a_cf, b_cf, d_cf = line_second_result["model"]

print("CIRCLE-FIRST FITTING RESULTS")
print("=" * 45)

print("\nCircle estimated first:")
print(f"Center = ({xc_cf:.4f}, {yc_cf:.4f})")
print(f"Radius = {r_cf:.4f}")
print(f"Inliers = {circle_first_result['count']}")

print("\nLine estimated second:")
print(f"a = {a_cf:.4f}")
print(f"b = {b_cf:.4f}")
print(f"d = {d_cf:.4f}")
print(f"Inliers = {line_second_result['count']}")
```

```
CIRCLE-FIRST FITTING RESULTS
=============================================

Circle estimated first:
Center = (1.8693, 3.1217)
Radius = 9.8279
Inliers = 54

Line estimated second:
a = -0.7195
b = -0.6945
d = -1.4827
Inliers = 36
```

2(d) — Reverse the order: circle first

To test order dependence, the circle is now fitted to **all 100 points** before its consensus is removed and a line is estimated from the remainder. The same thresholds and iteration budgets are used in both orders. This controls several algorithmic settings, although the random samples necessarily differ because the input point sets differ.

```
# ============================================
# Comparison: Line First vs Circle First
# ============================================
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 8))

def plot_fitting_result(ax, points, line_model,
                        circle_model, title,
                        line_points, circle_points):

    ax.scatter(
        points[:, 0], points[:, 1],
        s=20, color="lightgray",
        label="All points"
    )

    ax.scatter(
        line_points[:, 0], line_points[:, 1],
        s=30, color="orange",
        label="Line inliers"
    )

    ax.scatter(
        circle_points[:, 0], circle_points[:, 1],
        s=30, color="cyan",
        label="Circle inliers"
    )

    x_vals = np.linspace(-15, 15, 300)

    # Ground truth line
    ax.plot(
        x_vals, m_gt * x_vals + b_gt,
        "g--", linewidth=1.5,
        label="Ground truth line"
    )

    # Estimated line
    plot_line(
        ax, line_model, x_vals,
        color="blue", linewidth=2,
        label="Estimated line"
    )

    # Ground truth circle
    ax.add_patch(
        plt.Circle(
            (x0_gt, y0_gt), r_gt,
            fill=False, edgecolor="green",
            linestyle="--", linewidth=1.5,
            label="Ground truth circle"
        )
    )
```

```
    # Estimated circle
    xc, yc, r = circle_model

    ax.add_patch(
        plt.Circle(
            (xc, yc), r,
            fill=False, edgecolor="blue",
            linewidth=2,
            label="Estimated circle"
        )
    )

    ax.set_xlim(-15, 17)
    ax.set_ylim(-15, 17)
    ax.set_aspect("equal")
    ax.set_xlabel("X coordinate")
    ax.set_ylabel("Y coordinate")
    ax.set_title(title)
    ax.grid(alpha=0.2)
    ax.legend(fontsize=8)


# Line-first results
plot_fitting_result(
    axes[0],
    X,
    line_result["model"],
    circle_result["model"],
    "Method 1: Line First",
    X[line_result["inliers"]],
    X_remaining[circle_result["inliers"]]
)

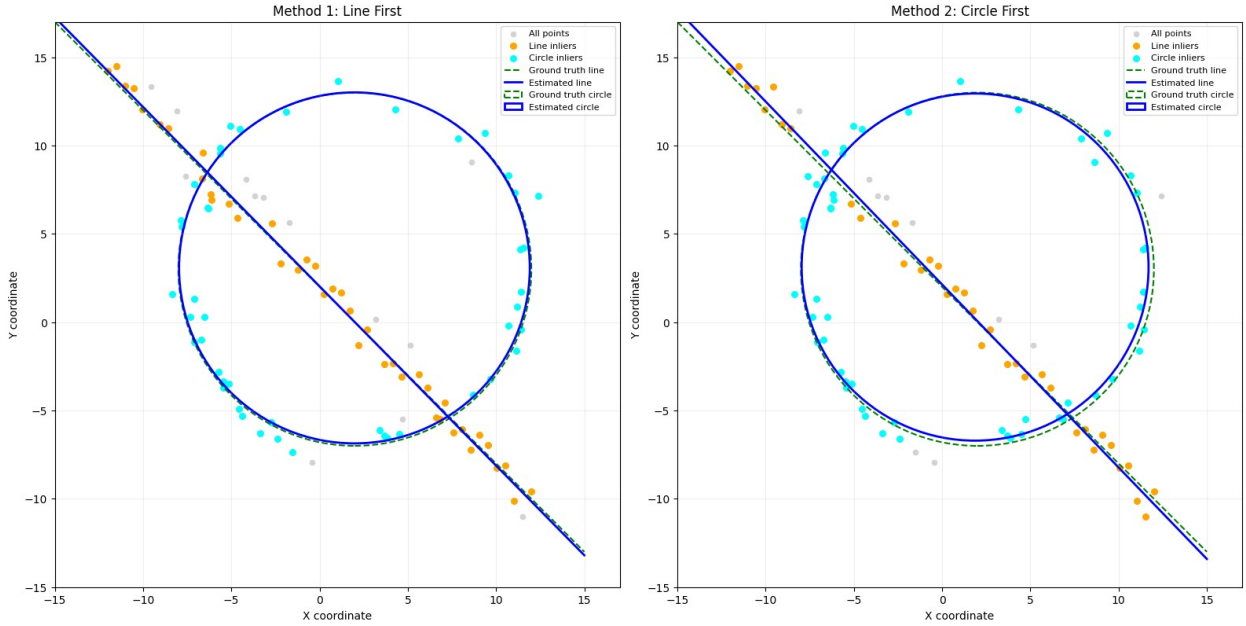
# Circle-first results
plot_fitting_result(
    axes[1],
    X,
    line_second_result["model"],
    circle_first_result["model"],
    "Method 2: Circle First",
    X_after_circle[line_second_result["inliers"]],
    X[circle_first_result["inliers"]]
)

plt.tight_layout()

plt.savefig(
    "Q2_Fitting_Order_Comparison.png",
    dpi=300,
    bbox_inches="tight"
```

```
)

plt.show()
```



Comparing the two fitting orders

The side-by-side plots allow visual inspection of model alignment and consensus assignment. Where the true circle approaches or intersects the line, points from one structure can fall within the other structure's residual threshold. Whichever model is fitted first can claim these ambiguous observations, changing the second model's available data. Neither fitting order is guaranteed to be superior in every random realization.

```
import pandas as pd

def calculate_line_errors(model):
    a, b, d = model

    normal_est = np.array([a, b])
    normal_gt = np.array([-m_gt, 1.0])
    normal_gt /= np.linalg.norm(normal_gt)

    d_gt = b_gt / np.sqrt(m_gt**2 + 1)

    if np.dot(normal_est, normal_gt) < 0:
        normal_est = -normal_est
        d = -d

    angle = np.degrees(
        np.arccos(
```

```
            np.clip(
                np.dot(normal_est, normal_gt),
                -1, 1
            )
        )
    )

    distance = abs(d - d_gt)

    return angle, distance


def calculate_circle_errors(model):
    xc, yc, r = model

    center_error = np.hypot(
        xc - x0_gt,
        yc - y0_gt
    )

    radius_error = abs(r - r_gt)

    return center_error, radius_error


# Method 1
line_angle_1, line_dist_1 = calculate_line_errors(
    line_result["model"]
)

circle_center_1, circle_radius_1 = calculate_circle_errors(
    circle_result["model"]
)

# Method 2
line_angle_2, line_dist_2 = calculate_line_errors(
    line_second_result["model"]
)

circle_center_2, circle_radius_2 = calculate_circle_errors(
    circle_first_result["model"]
)

comparison = pd.DataFrame({
    "Metric": [
        "Line angular error (degrees)",
        "Line distance error",
        "Circle center error",
        "Circle radius error",
        "Line inliers",
```

```
        "Circle inliers"
    ],
    "Line First": [
        line_angle_1,
        line_dist_1,
        circle_center_1,
        circle_radius_1,
        line_result["count"],
        circle_result["count"]
    ],
    "Circle First": [
        line_angle_2,
        line_dist_2,
        circle_center_2,
        circle_radius_2,
        line_second_result["count"],
        circle_first_result["count"]
    ]
})

display(comparison.round(4))

comparison.to_csv(
    "Q2_Fitting_Order_Comparison.csv",
    index=False
)
```


```
{"summary":"{\n  \"name\": \")\",\n  \"rows\": 6,\n  \"fields\": [\n
{\n      \"column\": \"Metric\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 6,\n
\"samples\": [\n          \"Line angular error (degrees)\",\n
\"Line distance error\",\n          \"Circle inliers\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Line First\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
22.44397556257952,\n        \"min\": 0.0113,\n        \"max\": 46.0,\n
\"num_unique_values\": 6,\n        \"samples\": [\n          0.4318,\n
0.0113,\n          46.0\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Circle First\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 23.74760976744116,\n
\"min\": 0.0685,\n        \"max\": 54.0,\n
\"num_unique_values\": 6,\n        \"samples\": [\n          1.0145,\n
0.0685,\n          54.0\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe"}
```

Discussion and conclusion

Sequential RANSAC can recover two geometric structures from a contaminated point set without relying on known point labels. The main design choices are the minimal sample size (two for a line, three for a circle), the geometric residual appropriate to each model, the consensus threshold, and the subsequent refinement. Evaluating against the known parameters distinguishes visual agreement from actual estimation accuracy.

Interpretation of fitting order. Fitting the circle first is not inherently invalid. It may obtain a good circle on the original cloud, but can absorb line points near the circular boundary; the later line then has fewer usable observations. Fitting the line first has the analogous risk of removing circle points near the line. Use the numerical comparison table above to state which order performed better in this specific fixed-seed trial rather than claiming a universal winner.

Limitations. This is a single simulated realization with fixed thresholds. The inlier masks are not reassessed after least-squares refinement, and the noise distributions differ between models. A broader reliability study would repeat the experiment across random seeds and thresholds and report variability of estimation errors.

Q2 additional checks: final-model residuals and consensus stability

The RANSAC masks stored in the results refer to the **winning minimal-sample hypotheses**, while the displayed final models have subsequently been refined. The distinction is intentional: Q2(c) asks for both the winning sample models and the best-fit models. The following diagnostic recomputes masks against the final models without overwriting the original RANSAC selections.

The synthetic line has vertical Gaussian noise of standard deviation 1, corresponding to an approximate perpendicular standard deviation of $1/\sqrt{2}$ for a slope of -1. Thus the line threshold 0.8 is about 1.13 perpendicular standard deviations. Circle noise has radial standard deviation $10/16 = 0.625$; threshold 1.0 is about 1.6 radial standard deviations. The 3,000 and 5,000 iterations are conservative choices: with an inlier probability of roughly 0.5, the standard RANSAC bound $N \geq \log(1-p)/\log(1-w^s)$ for $p = 0.99$ gives about 17 line samples ($s = 2$) and 35 circle samples ($s = 3$). Actual probabilities vary as models are removed and candidate sets change.

```
# Diagnostic only: original hypothesis consensus is preserved for all
prior plots.
line_refined_mask = line_errors(X, line_result["model"]) < 0.8
circle_refined_mask = circle_errors(X_remaining,
circle_result["model"]) < 1.0
print("Line: winning-hypothesis consensus =", line_result["count"],
      "; refined-model consensus =", int(line_refined_mask.sum()))
print("Circle: winning-hypothesis consensus =",
circle_result["count"],
      "; refined-model consensus =", int(circle_refined_mask.sum()))
print("Line normal magnitude =", np.linalg.norm(line_result["model"]
[:2]))
```

```
assert np.isclose(np.linalg.norm(line_result["model"][:2]), 1.0)
assert circle_result["model"][2] > 0
```

```
Line: winning-hypothesis consensus = 41 ; refined-model consensus = 40
Circle: winning-hypothesis consensus = 46 ; refined-model consensus =
46
Line normal magnitude = 1.0
```

Final Q2 conclusion

The line-first pipeline uses a two-point line hypothesis and perpendicular-distance consensus, removes that consensus, then applies three-point circle hypotheses with radial-distance consensus and least-squares refinement. The reverse-order experiment repeats the fitting procedure with the circle first. Compare **parameter errors**, not just the raw inlier counts: a larger consensus does not automatically imply a model closer to ground truth, especially where the two shapes intersect and share spatially nearby points. The outcome is dataset- and threshold-dependent; neither ordering is guaranteed to be superior for every realization.

Question 3 — Planar homography and image compositing

1. Input images and reproducibility

All four input files are stored in the GitHub `images/` folder. The loader downloads each file from the raw GitHub URL if no readable local copy is available. Thus, the notebook works in a fresh Google Colab runtime without manual uploads.

Coordinates: Image points use `(x, y)` in pixels, with the origin at the top-left. Each four-point list is ordered **top-left, top-right, bottom-right, bottom-left**.

```
from pathlib import Path
from urllib.request import Request, urlopen
import cv2
import numpy as np
import matplotlib.pyplot as plt

ROOT = Path.cwd()
IMAGE_DIR = ROOT / "images"
OUTPUT_DIR = ROOT / "results"
IMAGE_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

GITHUB_RAW = ("https://raw.githubusercontent.com/"
```

```
"NisalJayaweera/EN3160-Fitting-and-Alignment/main/images/")
BACKGROUND_PATH = IMAGE_DIR / "bgmain-mldlys04.png"
LOGO_PATH = IMAGE_DIR / "University_of_Moratuwa_logo.png"

def ensure_github_image(path: Path, flags: int):
    """Read a cached image or download and decode it from GitHub."""
    img = cv2.imread(str(path), flags) if path.is_file() else None
    if img is not None:
        print(f"Using cached image: {path}")
        return img
    url = GITHUB_RAW + path.name
    print(f"Downloading from GitHub: {url}")
    try:
        req = Request(url, headers={"User-Agent": "EN3160-assignment-
notebook"})
        with urlopen(req, timeout=45) as response:
            data = response.read()
    except Exception as exc:
        raise RuntimeError(f"Unable to download {path.name} from
{url}: {exc}") from exc
    decoded = cv2.imdecode(np.frombuffer(data, dtype=np.uint8), flags)
    if decoded is None:
        raise ValueError(f"Downloaded data is not a valid image:
{url}")
    path.write_bytes(data)
    return decoded

background = ensure_github_image(BACKGROUND_PATH, cv2.IMREAD_COLOR)
logo = ensure_github_image(LOGO_PATH, cv2.IMREAD_UNCHANGED)
if logo.ndim != 3 or logo.shape[2] != 4:
    raise ValueError("The source logo must be an RGBA PNG with
transparency.")
print("Background (H, W, C):", background.shape)
print("Logo (H, W, C):", logo.shape)

fig, axs = plt.subplots(1, 2, figsize=(14, 5))
axs[0].imshow(cv2.cvtColor(background, cv2.COLOR_BGR2RGB))
axs[0].set_title("Target image: university building")
axs[1].imshow(cv2.cvtColor(logo, cv2.COLOR_BGRA2RGBA))
axs[1].set_title("Source image: university crest")
for ax in axs: ax.axis("off")
plt.tight_layout()
plt.show()
```

```
Using cached image:
/content/EN3160-Fitting-and-Alignment/images/bgmain-mldlys04.png
Using cached image:
/content/EN3160-Fitting-and-Alignment/images/University_of_Moratuwa_lo
go.png
```

```
Background (H, W, C): (813, 1535, 3)
Logo (H, W, C): (325, 306, 4)
```

Target image: university building



Source image: university crest



2. Four-point correspondence

The source points are the four corners of the logo raster. The destination points are manually selected points on the approximately planar front fascia, in the same order: top-left, top-right, bottom-right, bottom-left. These pixel coordinates refer specifically to the supplied **1535 × 813** photograph. If the image changes resolution, reselect the points.

```
h, w = logo.shape[:2]
src_pts = np.float32([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]])
dst_pts = np.float32([[589, 327], [704, 322], [710, 420], [587, 415]])

preview = cv2.cvtColor(background.copy(), cv2.COLOR_BGR2RGB)
fig, ax = plt.subplots(figsize=(15, 7))
ax.imshow(preview)
polygon = np.vstack([dst_pts, dst_pts[0]])
ax.plot(polygon[:, 0], polygon[:, 1], 'c-', linewidth=2)
for i, (x, y) in enumerate(dst_pts):
    ax.scatter(x, y, c='red', s=35)
    ax.text(x + 5, y - 7, f'P{i+1}', color='white', fontsize=10,
bbox={'facecolor':'black','alpha':0.6,'pad':2})
ax.set_title('Manually selected destination quadrilateral')
ax.axis('off'); plt.tight_layout(); plt.show()
print('Source corners:\n', src_pts)
print('Destination corners:\n', dst_pts)
```

Manually selected destination quadrilateral




```
Source corners:
 [[  0.   0.]
 [305.   0.]
 [305. 324.]
 [  0. 324.]]
Destination corners:
 [[589. 327.]
 [704. 322.]
 [710. 420.]
 [587. 415.]]
```

3. Homography estimation

A homography maps a point (x, y) on the source plane to (x', y') on the target plane:

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, H \in R^{3\times 3}.$$

There are eight independent degrees of freedom because the matrix is defined up to scale. Four non-collinear point correspondences determine a homography. `cv2.getPerspectiveTransform` estimates it directly; RANSAC is unnecessary when there are exactly four manually selected correspondences. The reprojection check below tests whether the four source corners map to the intended target corners.

```
H = cv2.getPerspectiveTransform(src_pts, dst_pts)
projected = cv2.perspectiveTransform(src_pts.reshape(-1, 1, 2),
```

```
H).reshape(-1, 2)
corner_error = np.linalg.norm(projected - dst_pts, axis=1)
print('Homography H:\n', H)
print('Corner reprojection errors (pixels):', np.round(corner_error,
6))
assert np.all(np.isfinite(H)) and np.max(corner_error) < 1e-2
```

```
Homography H:
 [[ 1.48609884e-01 -1.15262274e-01  5.89000000e+02]
 [-1.20878462e-01  1.94480381e-01  3.27000000e+02]
 [-3.24487637e-04 -1.85842307e-04  1.00000000e+00]]
Corner reprojection errors (pixels): [0. 0. 0. 0.]
```

4. Perspective warping and alpha compositing — pair 1

The transparent logo is warped with `cv2.warpPerspective`. Its alpha channel is used to blend it into the photograph:

$$I_{out}(x,y) = \alpha(x,y)\, I_{warp}(x,y) + \left[1-\alpha(x,y)\right] I_{background}(x,y).$$

We apply an additional opacity factor of 0.45. A warped alpha value of zero leaves the background unchanged.

```
bh, bw = background.shape[:2]
warped = cv2.warpPerspective(logo, H, (bw, bh),
flags=cv2.INTER_LINEAR,
                             borderMode=cv2.BORDER_CONSTANT,
borderValue=(0, 0, 0, 0))
opacity = 0.45
alpha = (warped[:, :, 3:4].astype(np.float32) / 255.0) * opacity
composite = (alpha * warped[:, :, :3].astype(np.float32)
             + (1 - alpha) * background.astype(np.float32))
composite = np.clip(composite, 0, 255).astype(np.uint8)
cv2.imwrite(str(OUTPUT_DIR / 'q3_pair1_composite.png'), composite)
cv2.imwrite(str(OUTPUT_DIR / 'q3_pair1_warped_logo.png'), warped)

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes[0, 0].imshow(cv2.cvtColor(background, cv2.COLOR_BGR2RGB));
axes[0, 0].set_title('Original photograph')
axes[0, 1].imshow(cv2.cvtColor(logo, cv2.COLOR_BGRA2RGBA)); axes[0,
1].set_title('Source crest')
axes[1, 0].imshow(cv2.cvtColor(warped, cv2.COLOR_BGRA2RGBA)); axes[1,
0].set_title('Perspective-warped crest (RGBA)')
axes[1, 1].imshow(cv2.cvtColor(composite, cv2.COLOR_BGR2RGB)); axes[1,
1].set_title('Final alpha-composited result')
for ax in axes.flat: ax.axis('off')
plt.tight_layout(); plt.savefig(OUTPUT_DIR /
'q3_pair1_comparison.png', dpi=160); plt.show()
```

Original photograph



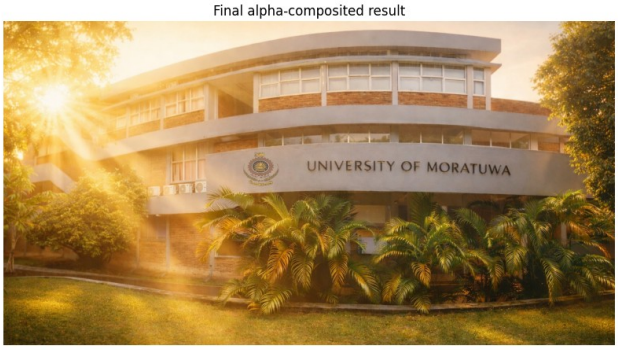
Source crest



Perspective-warped crest (RGBA)



Final alpha-composited result



5. Pair 1 discussion — university crest on the façade

The building's pale front fascia provides a relatively smooth, visually suitable placement area, and the chosen polygon avoids the prominent existing lettering. The crest's transparent background prevents a rectangular patch from covering the building. The four correspondences are fitted essentially exactly; the reported **corner reprojection error only checks the four chosen correspondences**, not the realism of the composite.

The main limitation is that the fascia is slightly **curved**. A homography describes a plane, so it only approximates the surface locally. The overlay also does not model wall texture, shadows, or environmental lighting. A piecewise warp or surface-aware projection could improve realism.

6. Image pair 2 — Sri Lankan flag on the Sigiriya rock face

Source: `Flag_of_Sri_Lanka.svg.webp` (opaque flag illustration).
Target: `Sigiriya+Rock.webp` (photograph of Sigiriya).

The rock face is a meaningful outdoor surface for a virtual flag projection. We choose a broad, approximately flat **local patch** of the visible rock face rather than projecting across the entire irregular rock. The selected destination quadrilateral gives a perspective effect. Unlike the transparent university crest, the flag is opaque, so a warped polygon mask and mild feathering are used to blend its boundary.

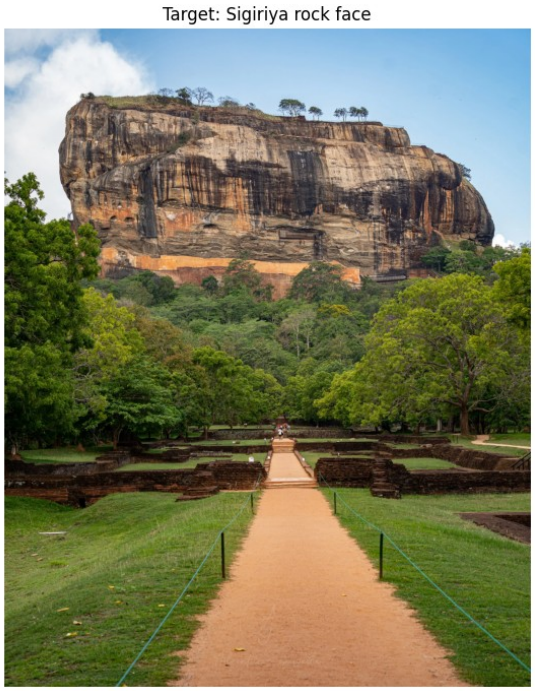
Important limitation: Sigiriya's rock surface is three-dimensional and uneven, not a perfect plane. The homography is therefore an approximation for this demonstration, not a physically exact texture map.

```
FLAG_PATH = IMAGE_DIR / "Flag_of_Sri_Lanka.svg.webp"
SIGIRIYA_PATH = IMAGE_DIR / "Sigiriya+Rock.webp"
flag = ensure_github_image(FLAG_PATH, cv2.IMREAD_COLOR)
sigiriya = ensure_github_image(SIGIRIYA_PATH, cv2.IMREAD_COLOR)
print("Flag:", flag.shape, "| Sigiriya:", sigiriya.shape)

fig, ax = plt.subplots(1, 2, figsize=(13, 7))
ax[0].imshow(cv2.cvtColor(flag, cv2.COLOR_BGR2RGB))
ax[0].set_title("Source: Sri Lankan flag")
ax[1].imshow(cv2.cvtColor(sigiriya, cv2.COLOR_BGR2RGB))
ax[1].set_title("Target: Sigiriya rock face")
for a in ax: a.axis("off")
plt.tight_layout()
plt.show()
```

```
Using cached image:
/content/EN3160-Fitting-and-Alignment/images/Flag_of_Sri_Lanka.svg.webp
Using cached image:
/content/EN3160-Fitting-and-Alignment/images/Sigiriya+Rock.webp
Flag: (1920, 3840, 3) | Sigiriya: (3125, 2500, 3)
```





6.1 Four-point correspondences

The source quadrilateral consists of the flag's image corners. Destination points were selected manually within the rock's broad central face. The order of points must be consistent between the two sets. These manually selected coordinates are **design choices for the virtual projection**, not measured physical landmarks.

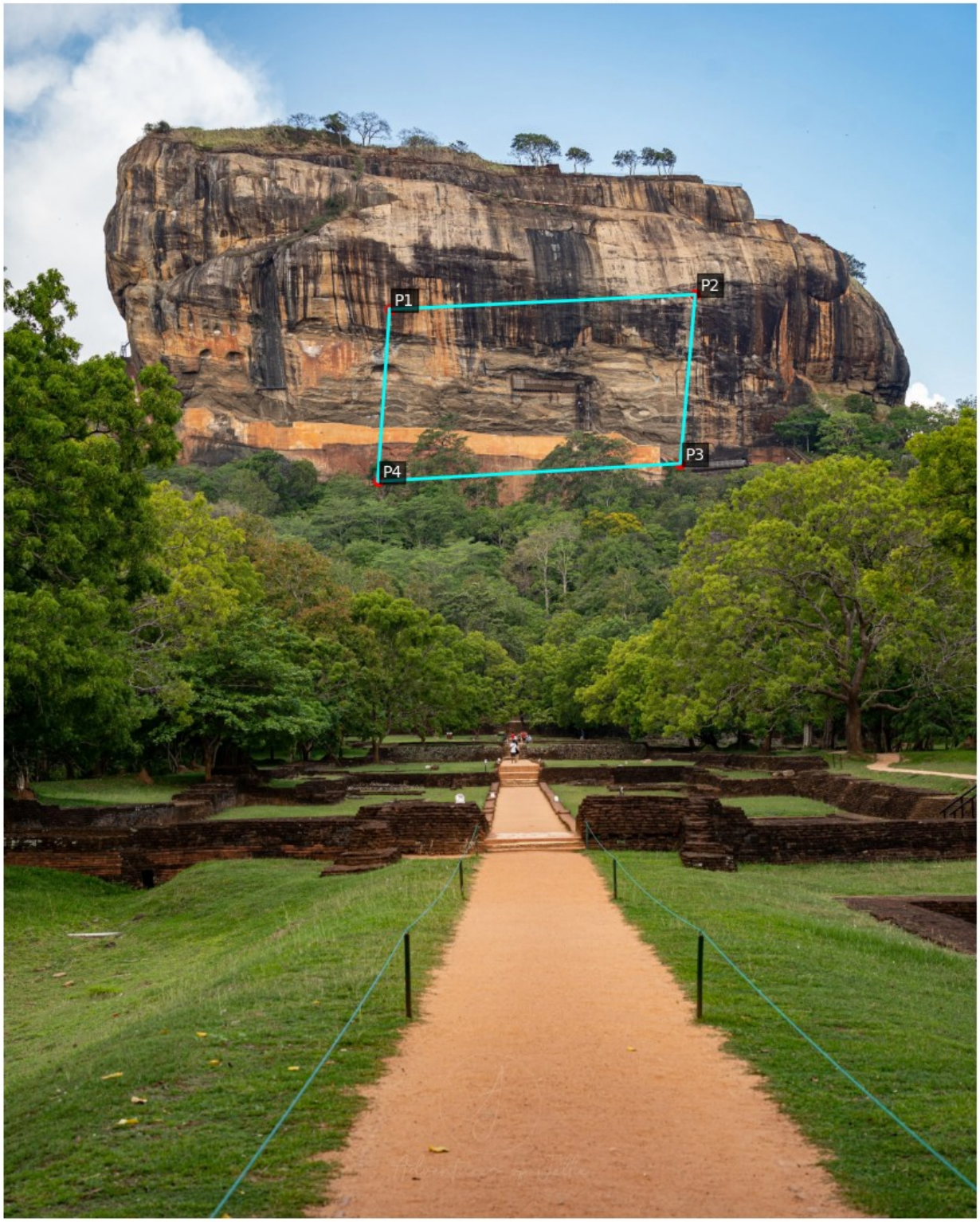
The destination polygon is specified relative to a reference image size and scaled to the actual loaded photograph. This removes the fixed-resolution assumption. **If the image is cropped or otherwise reframed**, inspect the displayed polygon and adjust the destination points manually.

```
fh, fw = flag.shape[:2]
flag_src = np.float32([[0, 0], [fw - 1, 0], [fw - 1, fh - 1], [0, fh -
1]])
# Destination quadrilateral defined in the reference photo
(width=1638, height=2048).
# Scale coordinates to the ACTUAL downloaded image dimensions.
# This supports resolution changes when the photo content and crop are
unchanged.
reference_width, reference_height = 1638, 2048
reference_dst = np.float32([[650, 515], [1165, 490], [1140, 775],
[630, 805]])
actual_height, actual_width = sigiriya.shape[:2]
scale_xy = np.float32([actual_width / reference_width,
                       actual_height / reference_height])
flag_dst = reference_dst * scale_xy
print(f"Sigiriya image size: {actual_width} x {actual_height}")
print(f"Destination point scaling: x={scale_xy[0]:.4f},
y={scale_xy[1]:.4f}")

fig, ax = plt.subplots(figsize=(10, 11))
ax.imshow(cv2.cvtColor(sigiriya, cv2.COLOR_BGR2RGB))
outline = np.vstack([flag_dst, flag_dst[0]])
ax.plot(outline[:, 0], outline[:, 1], color="cyan", lw=2)
for i, (x, y) in enumerate(flag_dst):
    ax.scatter(x, y, color="red", s=35)
    ax.text(x+12, y-12, f"P{i+1}", color="white",
            bbox={"facecolor":"black", "alpha":0.65, "pad":2})
ax.set_title("Pair 2: chosen rock-face destination quadrilateral")
ax.axis("off")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "q3_pair2_correspondences.png", dpi=130,
bbox_inches="tight")
plt.show()
print("Source corners:\n", flag_src)
print("Destination corners:\n", flag_dst)
```

```
Sigiriya image size: 2500 x 3125
Destination point scaling: x=1.5263, y=1.5259
```

Pair 2: chosen rock-face destination quadrilateral




```
Source corners:
 [[   0.    0.]
```

```
 [3839.    0.]
 [3839. 1919.]
 [   0. 1919.]]
Destination corners:
 [[ 992.06354  785.82764]
 [1778.083    747.68066]
 [1739.9268  1182.5562 ]
 [ 961.53845 1228.3325 ]]
```

6.2 Homography estimation and geometric validation

For a source point (\mathbf{x}=(x,y,1)^T) and a target point (\mathbf{x}'), the projective mapping satisfies (\lambda\mathbf{x}'=H\mathbf{x}). Four non-collinear correspondences determine the eight independent degrees of freedom of (H) (up to scale). `cv2.getPerspectiveTransform` computes this mapping. A near-zero corner error confirms numerical agreement at the four specified points, **not** accurate mapping of an uneven surface.

```
H_flag = cv2.getPerspectiveTransform(flag_src, flag_dst)
mapped_corners = cv2.perspectiveTransform(flag_src.reshape(-1, 1, 2),
H_flag).reshape(-1, 2)
errors = np.linalg.norm(mapped_corners - flag_dst, axis=1)
print("Pair 2 homography H:\n", H_flag)
print("Corner reprojection errors (px):", np.round(errors, 6))
assert np.all(np.isfinite(H_flag)) and np.max(errors) < 0.01
```

```
Pair 2 homography H:
 [[ 2.13394156e-01 -1.05358300e-02  9.92063538e+02]
 [-6.30011499e-03  2.37452581e-01  7.85827637e+02]
 [ 4.86381474e-06  5.58577440e-06  1.00000000e+00]]
Corner reprojection errors (px): [0. 0. 0. 0.]
```

6.3 Warping and masked blending

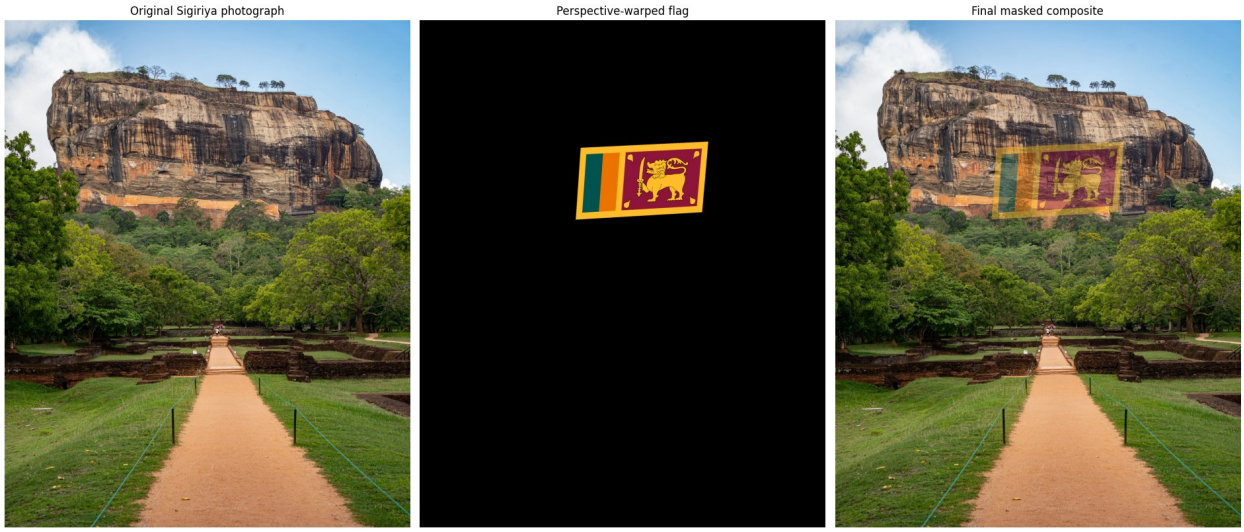
The flag is warped into the photograph's coordinate system. We also warp a white source mask, which determines the valid projected quadrilateral. A small Gaussian blur softens the mask boundary; an opacity factor of 0.45 allows some underlying rock texture to remain visible. The final image is a **visual composite**, not a physically based projection.

```
sh, sw = sigiriya.shape[:2]
flag_warped = cv2.warpPerspective(flag, H_flag, (sw, sh),
                                  flags=cv2.INTER_LINEAR,
                                  borderMode=cv2.BORDER_CONSTANT,
                                  borderValue=(0, 0, 0))
source_mask = np.full((fh, fw), 255, dtype=np.uint8)
warped_mask = cv2.warpPerspective(source_mask, H_flag, (sw, sh),
                                   flags=cv2.INTER_LINEAR,
                                   borderMode=cv2.BORDER_CONSTANT,
                                   borderValue=0)
```

```
soft_mask = cv2.GaussianBlur(warped_mask, (7, 7), 1.4)
blend_alpha = 0.45 * (soft_mask.astype(np.float32) / 255.0)[..., None]
flag_composite = np.clip(blend_alpha * flag_warped.astype(np.float32)
                         + (1.0 - blend_alpha) *
sigiriya.astype(np.float32),
                         0, 255).astype(np.uint8)

cv2.imwrite(str(OUTPUT_DIR / "q3_pair2_composite.png"),
flag_composite)
cv2.imwrite(str(OUTPUT_DIR / "q3_pair2_warped_flag.png"), flag_warped)
cv2.imwrite(str(OUTPUT_DIR / "q3_pair2_projection_mask.png"),
warped_mask)

fig, ax = plt.subplots(1, 3, figsize=(19, 9))
for a, image, title in zip(ax,
    [sigiriya, flag_warped, flag_composite],
    ["Original Sigiriya photograph", "Perspective-warped flag", "Final
masked composite"]):
    a.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    a.set_title(title)
    a.axis("off")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "q3_pair2_comparison.png", dpi=140,
bbox_inches="tight")
plt.show()
```



7. Discussion and comparison of the two image pairs

Aspect	Pair 1: University crest	Pair 2: Flag at Sigiriya
Source transparency	Transparent RGBA PNG	Opaque RGB WebP
Target region	Pale façade / fascia	Central rock face
Geometric model	Four-point planar	Four-point planar

Aspect	Pair 1: University crest	Pair 2: Flag at Sigiriya
	homography	homography
Blending	Warped source alpha × 0.45 opacity	Warped polygon mask × 0.45 opacity with feathering
Primary limitation	Curved fascia, lighting	Uneven rock depth, lighting and texture

Both experiments demonstrate how a homography maps a rectangular graphic to a chosen quadrilateral in another image. The correspondences were selected manually because the task is to **place** a new graphic on a surface, rather than find pre-existing matching features between the two unrelated images.

The first experiment benefits from a transparent source and a relatively small target patch. The second uses an opaque source, so masking is essential to prevent the rest of the source rectangle from obscuring the scene. Both use alpha compositing to preserve some context.

Geometric accuracy: The nearly zero corner reprojection error follows from fitting exactly four points. It does not measure physical accuracy on a non-planar façade or rock face. Straight lines in the graphic remain straight after a homography, but they will not conform to a curved or irregular surface.

Photometric realism: Neither experiment estimates surface reflectance, occlusion, cast shadows, or spatially varying illumination. The resulting overlays demonstrate projective alignment and compositing, not photorealistic augmented reality.

Conclusion: The two image pairs meet the requested four-point correspondence, homography estimation, image warping, and visual-compositing workflow. A surface-aware mesh warp and illumination-aware blending would be appropriate improvements for more realistic results.
Resolution handling: The Sigiriya destination points are specified in a 1638 × 2048 reference coordinate system and scaled to the loaded photograph. This compensates for resizing **without cropping**; a different crop or perspective requires reselecting the four destination points.

Question 4 — SIFT PCB alignment and difference analysis

EN3160 — Assignment 02 | Question 4

PCB image registration and difference analysis using SIFT + RANSAC

Reference: `AECC1.jpg` · **Test images:** `AECC2.jpg`, `AECC3.jpg`, `AECC4.jpg`, `AAAA1.jpg`, `ADAD1.jpg`

This notebook implements all parts **4(a)–4(e)** of the assignment. It downloads the six original PCB photographs from the `main/images/` folder of the GitHub repository, estimates projective mappings into the template coordinate system, and visualizes the differences.

Reproducibility: Run all cells in order in Google Colab or a local Jupyter environment with internet access for the initial download. All figures and a metrics CSV are saved to `results/q4/`.

Method and geometry

1. **4(a):** Detect SIFT keypoints and 128-dimensional descriptors in both grayscale images.
2. **4(b):** Find tentative matches with `cv2.BFMatcher(cv2.NORM_L2)` and Lowe's ratio test; estimate a homography with `cv2.findHomography(..., cv2.RANSAC, ...)`. Draw inlier correspondences.
3. **4(c):** Warp each *test image into the template frame* with `cv2.warpPerspective`. Warp an all-ones mask too, so invalid border pixels are excluded from comparison.
4. **4(d):** Repeat for all five test photographs, show and save each difference image.
5. **4(e):** Discuss why tall or changed components (USB Type-B connector, capacitors, reset switch) produce large residuals.

For corresponding points in homogeneous coordinates, the estimated projective transformation is

[\lambda\begin{bmatrix}x_t\y_t\1\end{bmatrix} =H\begin{bmatrix}x_s\y_s\1\end{bmatrix}, \qquad H\in\mathbb{R}^{3\times3}.]

Here (s) is the **source/test** image and (t) is the **template** image. RANSAC estimates (H) robustly despite mismatched feature pairs. The difference shown is the absolute intensity residual (D(x,y)=|I_t(x,y)-I_{s\to t}(x,y)|), restricted to the warped image's valid overlap.

```
from pathlib import Path
import urllib.request
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print("OpenCV:", cv2.__version__)
assert hasattr(cv2, "SIFT_create"), "Install a version of OpenCV with
SIFT support."

REPO = "NisalJayaweera/EN3160-Fitting-and-Alignment"
BRANCH = "main"
IMAGE_DIR = Path("images")
RESULT_DIR = Path("results/q4")
IMAGE_DIR.mkdir(parents=True, exist_ok=True)
RESULT_DIR.mkdir(parents=True, exist_ok=True)

TEMPLATE_NAME = "AECC1.jpg"
TEST_NAMES = ["AECC2.jpg", "AECC3.jpg", "AECC4.jpg", "AAAA1.jpg",
```

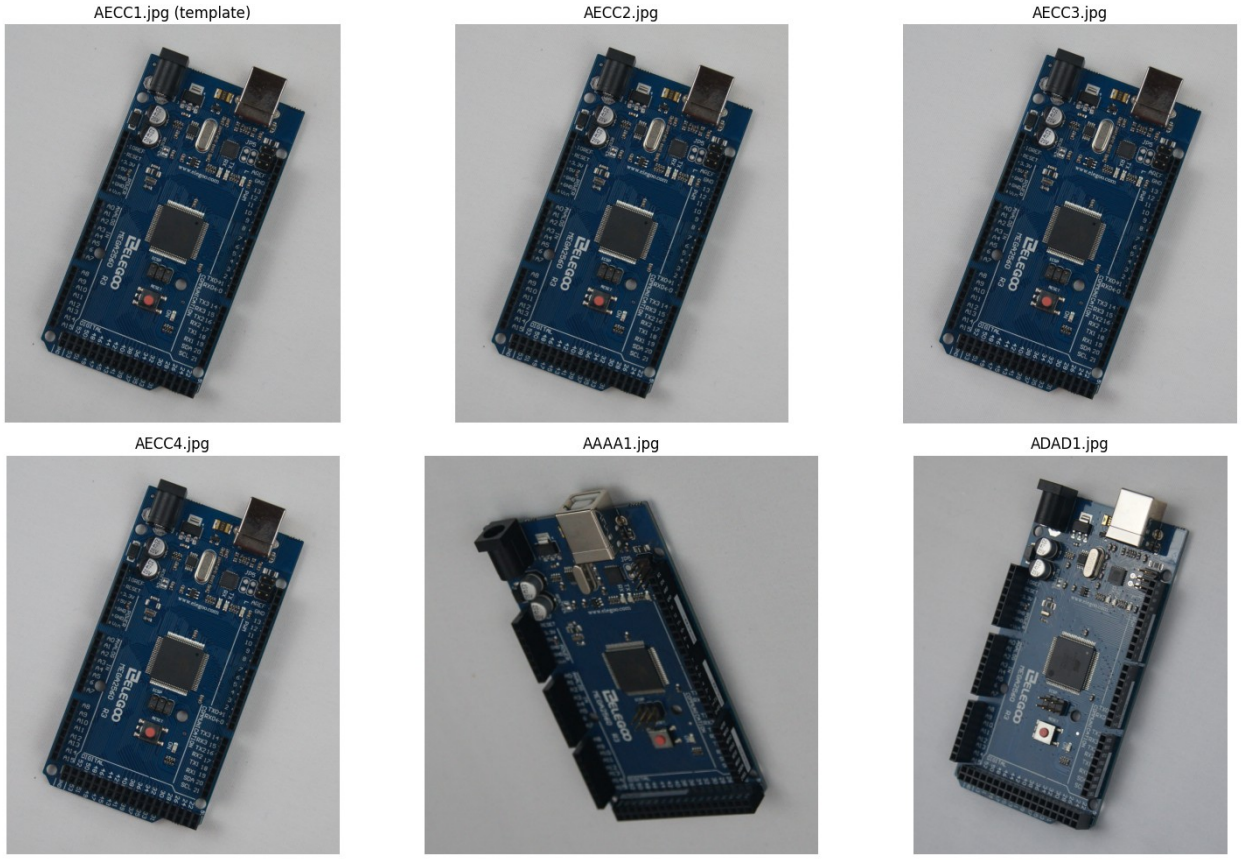
```
"ADAD1.jpg"]
ALL_NAMES = [TEMPLATE_NAME] + TEST_NAMES

def ensure_image(name):
    destination = IMAGE_DIR / name
    url =
f"https://raw.githubusercontent.com/{REPO}/{BRANCH}/images/{name}"
    if not destination.is_file():
        print("Downloading", url)
        try:
            urllib.request.urlretrieve(url, destination)
        except Exception as exc:
            raise RuntimeError(f"Could not download {name} from
GitHub: {url}") from exc
    img = cv2.imread(str(destination), cv2.IMREAD_COLOR)
    if img is None:
        raise RuntimeError(f"{destination} is not a readable image.
Check GitHub path and file contents.")
    return img

images = {name: ensure_image(name) for name in ALL_NAMES}
for name, img in images.items():
    print(f"{name:12s}: {img.shape[1]} x {img.shape[0]}")
```

```
OpenCV: 5.0.0
AECC1.jpg   : 2302 x 2730
AECC2.jpg   : 2279 x 2729
AECC3.jpg   : 2287 x 2731
AECC4.jpg   : 2281 x 2731
AAAA1.jpg   : 1956 x 1982
ADAD1.jpg   : 2169 x 2754
```

```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
for ax, name in zip(axes.flat, ALL_NAMES):
    ax.imshow(cv2.cvtColor(images[name], cv2.COLOR_BGR2RGB))
    ax.set_title(name + (" (template)" if name == TEMPLATE_NAME else
""))
    ax.axis("off")
plt.tight_layout()
plt.savefig(RESULT_DIR / "q4_input_images.png", dpi=150,
bbox_inches="tight")
plt.show()
```



4(a) SIFT features

SIFT extracts local scale- and rotation-invariant descriptors. To avoid excessive memory consumption on large photographs, feature extraction is performed on resized copies (maximum side `1400` pixels). All displayed alignments and difference images use this same common working resolution for consistency; the homography is therefore estimated in the working-image coordinates.

```
MAX_SIDE = 1400
def resize_for_analysis(image, max_side=MAX_SIDE):
    h, w = image.shape[:2]
    scale = min(1.0, max_side / max(h, w))
    if scale == 1.0:
        return image.copy()
    return cv2.resize(image, (round(w*scale), round(h*scale)),
interpolation=cv2.INTER_AREA)

working = {name: resize_for_analysis(im) for name, im in
images.items()}
grays = {name: cv2.cvtColor(im, cv2.COLOR_BGR2GRAY) for name, im in
working.items()}
sift = cv2.SIFT_create(nfeatures=4500)
features = {}
```

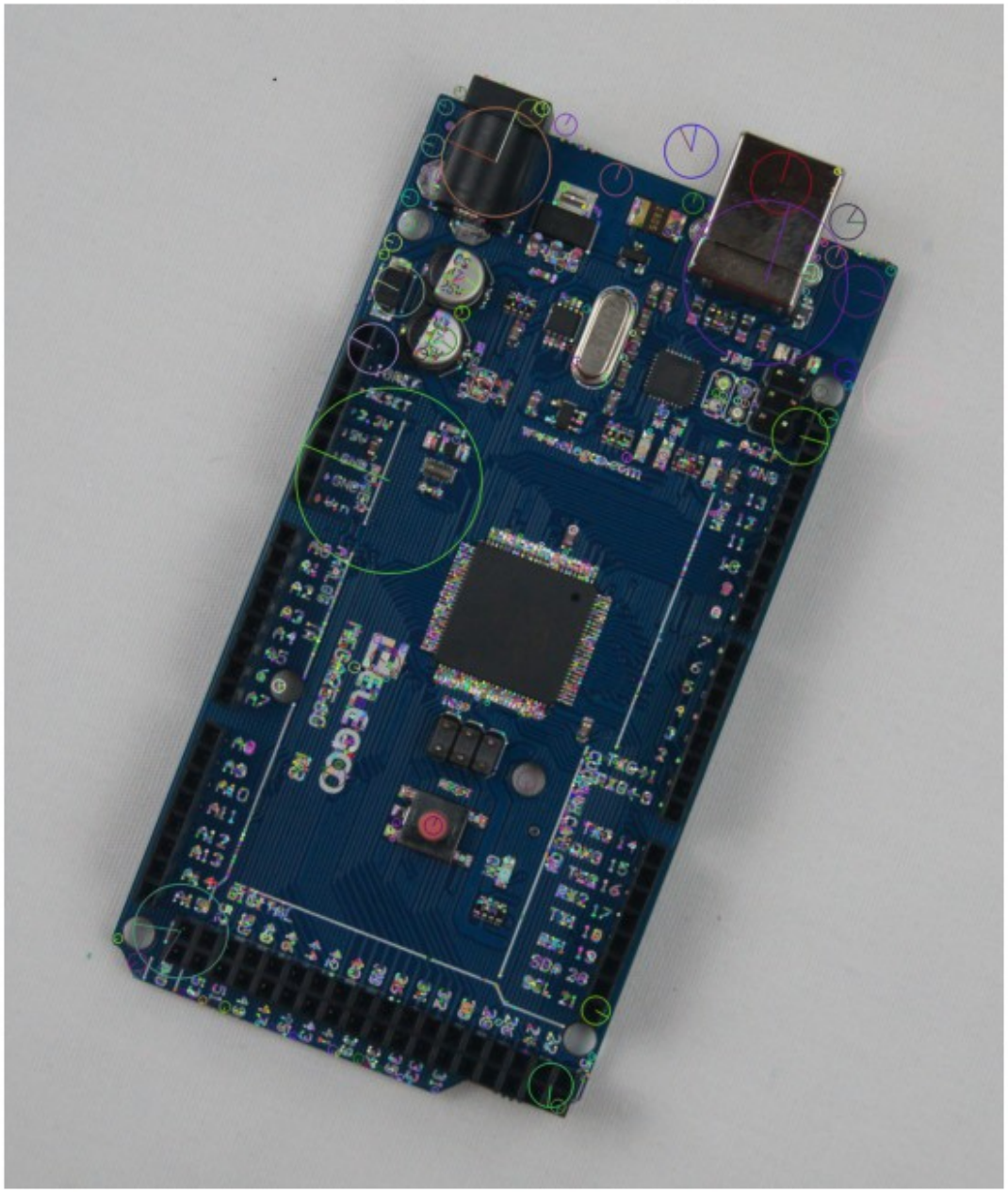
```
for name in ALL_NAMES:
    kp, desc = sift.detectAndCompute(grays[name], None)
    if desc is None or len(kp) < 4:
        raise RuntimeError(f"Insufficient SIFT features in {name}.")
    features[name] = (kp, desc)
    print(f"{name:12s}: {len(kp)} SIFT keypoints")

fig, ax = plt.subplots(figsize=(12, 7))
marked = cv2.drawKeypoints(working[TEMPLATE_NAME],
features[TEMPLATE_NAME][0], None,

flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
ax.imshow(cv2.cvtColor(marked, cv2.COLOR_BGR2RGB))
ax.set_title(f"SIFT keypoints — {TEMPLATE_NAME}")
ax.axis("off")
plt.tight_layout()
plt.savefig(RESULT_DIR / "q4_template_sift_keypoints.png", dpi=150,
bbox_inches="tight")
plt.show()
```

```
AECC1.jpg   : 4500 SIFT keypoints
AECC2.jpg   : 4500 SIFT keypoints
AECC3.jpg   : 4500 SIFT keypoints
AECC4.jpg   : 4500 SIFT keypoints
AAAA1.jpg   : 1941 SIFT keypoints
ADAD1.jpg   : 4500 SIFT keypoints
```

SIFT keypoints — AECC1.jpg



4(b) Feature matching and RANSAC homography

`knnMatch(k=2)` returns the two nearest descriptor matches. The ratio test rejects ambiguous descriptors. RANSAC then removes geometrically inconsistent matches, yielding a homography supported by inliers. A valid homography requires at least four correspondences; here we also enforce a modest inlier count and check the projected quadrilateral for degeneracy.

```
RATIO = 0.75
RANSAC_REPROJECTION_THRESHOLD = 4.0
```

```
matcher = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)

def align_and_compare(name):
    ref_name = TEMPLATE_NAME
    kp_src, des_src = features[name]
    kp_ref, des_ref = features[ref_name]

    pairs = matcher.knnMatch(des_src, des_ref, k=2)
    good = [m for pair in pairs if len(pair) == 2 for m, n in [pair]
if m.distance < RATIO*n.distance]
    if len(good) < 4:
        raise RuntimeError(f"{name}: only {len(good)} ratio-test
matches; need at least 4.")

    src_xy = np.float32([kp_src[m.queryIdx].pt for m in
good]).reshape(-1,1,2)
    dst_xy = np.float32([kp_ref[m.trainIdx].pt for m in
good]).reshape(-1,1,2)
    H, mask = cv2.findHomography(src_xy, dst_xy, cv2.RANSAC,

ransacReprojThreshold=RANSAC_REPROJECTION_THRESHOLD,
                                 maxIters=5000, confidence=0.995)
    if H is None or mask is None:
        raise RuntimeError(f"{name}: homography estimation failed.")
    inlier_mask = mask.ravel().astype(bool)
    n_inliers = int(inlier_mask.sum())
    if n_inliers < 8:
        raise RuntimeError(f"{name}: only {n_inliers} RANSAC inliers;
registration is unreliable.")

    src = working[name]
    ref = working[ref_name]
    h, w = ref.shape[:2]
    # Sanity-check the transformed source footprint before warping.
    sh, sw = src.shape[:2]
    corners = np.float32([[0, 0], [sw-1, 0], [sw-1, sh-1], [0, sh-
1]]).reshape(-1, 1, 2)
    projected = cv2.perspectiveTransform(corners, H).reshape(-1, 2)
    if not np.isfinite(projected).all():
        raise RuntimeError(f"{name}: homography projects corners to
non-finite coordinates.")
    polygon_area = abs(cv2.contourArea(projected.astype(np.float32)))
    if polygon_area < 0.01 * w * h or not
cv2.isContourConvex(projected.astype(np.float32)):
        raise RuntimeError(f"{name}: projected quadrilateral is
degenerate or non-convex.")
    aligned = cv2.warpPerspective(src, H, (w, h))
    overlap = cv2.warpPerspective(np.full(src.shape[:2], 255,
np.uint8), H, (w, h),
                                  flags=cv2.INTER_NEAREST) > 0
```

```
    # Avoid edge artifacts at the warped support boundary
    overlap = cv2.erode(overlap.astype(np.uint8),
np.ones((5,5),np.uint8), iterations=1).astype(bool)
    if overlap.sum() == 0:
        raise RuntimeError(f"{name}: no overlap with the template.")

    difference = cv2.absdiff(ref, aligned)
    difference[~overlap] = 0
    diff_gray = cv2.cvtColor(difference, cv2.COLOR_BGR2GRAY)
    mae = float(np.mean(difference[overlap]))
    gray_mae = float(np.mean(diff_gray[overlap]))
    overlap_fraction = float(np.mean(overlap))

    # For visualization, show only inlier matches; cap to 70 for
readability
    inlier_matches = [m for m, ok in zip(good, inlier_mask) if ok]
    shown = sorted(inlier_matches, key=lambda m: m.distance)[:70]
    match_image = cv2.drawMatches(src, kp_src, ref, kp_ref, shown,
None,

flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

    heat = cv2.applyColorMap(diff_gray, cv2.COLORMAP_JET)
    heat[~overlap] = 0
    return dict(name=name, H=H, matches=len(good), inliers=n_inliers,
                inlier_ratio=n_inliers/len(good), aligned=aligned,
                overlap=overlap, difference=difference,
diff_gray=diff_gray,
                heat=heat, match_image=match_image, mae=mae,
                gray_mae=gray_mae, overlap_fraction=overlap_fraction)
```

4(c–d) Register all PCB images and visualize differences

For each test image, the output shows the matched SIFT features, the template, the warped test photograph, the absolute difference, and a heatmap. **Black areas outside the overlap are invalid pixels, not perfect matches.** Residual brightness can arise from true PCB changes, lighting/exposure, parallax, occlusion, and imperfect registration.

```
results = {}
metrics = []
for name in TEST_NAMES:
    print(f"Processing {name} ...")
    try:
        result = align_and_compare(name)
    except Exception as exc:
        print(f"WARNING: {name} failed: {exc}")
        continue
    results[name] = result
    metrics.append({
```

```
        "image": name,
        "ratio_test_matches": result["matches"],
        "ransac_inliers": result["inliers"],
        "inlier_ratio": result["inlier_ratio"],
        "overlap_fraction": result["overlap_fraction"],
        "color_MAE_0_to_255": result["mae"],
        "gray_MAE_0_to_255": result["gray_mae"],
    })

    stem = Path(name).stem
    cv2.imwrite(str(RESULT_DIR / f"{stem}_aligned.jpg"),
result["aligned"])
    cv2.imwrite(str(RESULT_DIR / f"{stem}_difference.png"),
result["difference"])
    cv2.imwrite(str(RESULT_DIR / f"{stem}_difference_heatmap.png"),
result["heat"])
    cv2.imwrite(str(RESULT_DIR / f"{stem}_inlier_matches.jpg"),
result["match_image"])

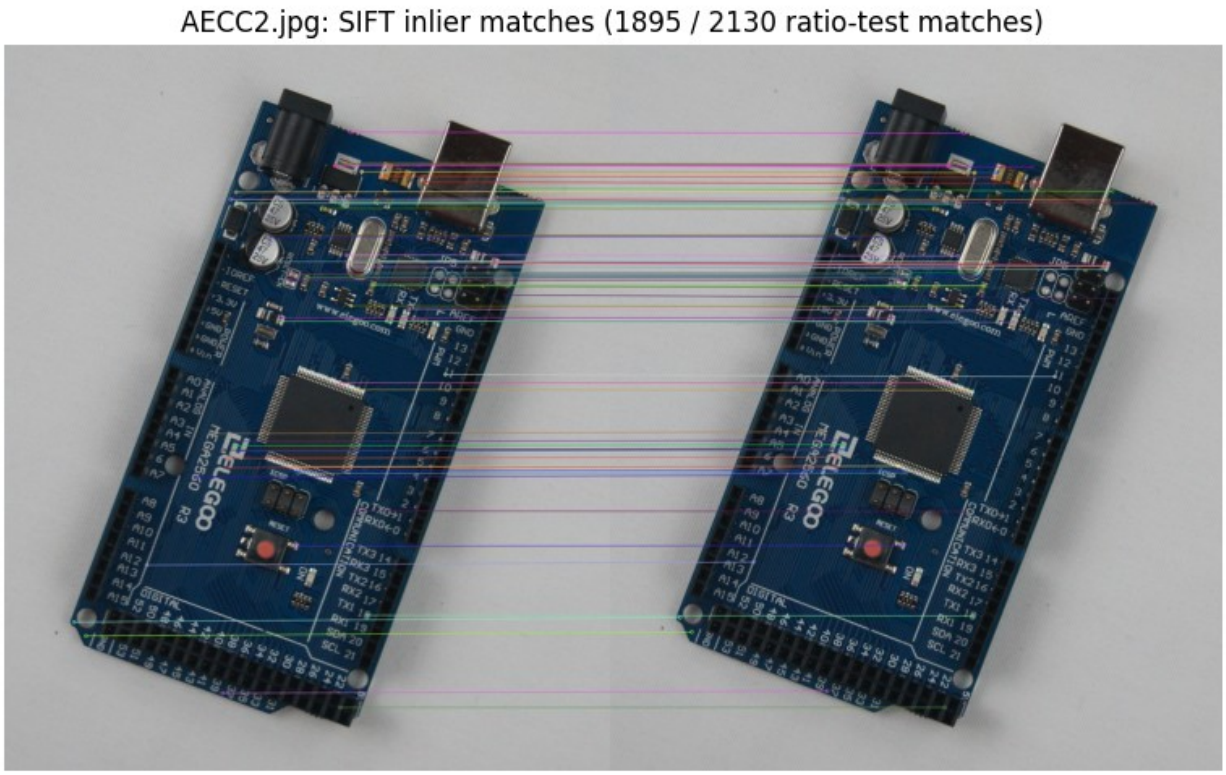
    fig, ax = plt.subplots(figsize=(15, 5))
    ax.imshow(cv2.cvtColor(result["match_image"], cv2.COLOR_BGR2RGB))
    ax.set_title(f"{name}: SIFT inlier matches ({result['inliers']} /
{result['matches']} ratio-test matches)")
    ax.axis("off")
    plt.tight_layout()
    plt.savefig(RESULT_DIR / f"{stem}_matches_figure.png", dpi=120,
bbox_inches="tight")
    plt.show()

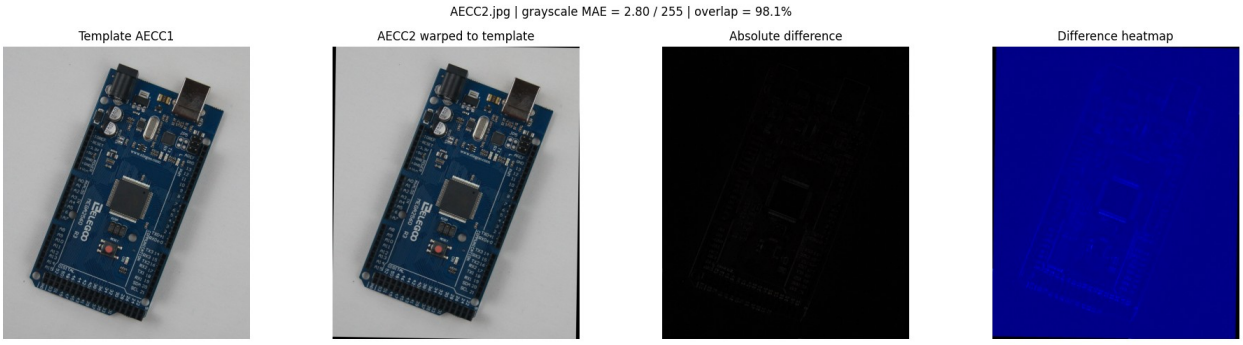
    fig, axs = plt.subplots(1, 4, figsize=(19, 5))
    panels = [
        (working[TEMPLATE_NAME], "Template AECC1"),
        (result["aligned"], f"{stem} warped to template"),
        (result["difference"], "Absolute difference"),
        (result["heat"], "Difference heatmap"),
    ]
    for ax, (img, title) in zip(axs, panels):
        ax.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        ax.set_title(title)
        ax.axis("off")
    fig.suptitle(f"{name} | grayscale MAE = {result['gray_mae']:.2f} /
255 | overlap = {result['overlap_fraction']:.1%}")
    plt.tight_layout()
    plt.savefig(RESULT_DIR / f"{stem}_comparison.png", dpi=140,
bbox_inches="tight")
    plt.show()

if len(results) != len(TEST_NAMES):
    print(f"Completed {len(results)} of {len(TEST_NAMES)} images;
review warnings above.")
```

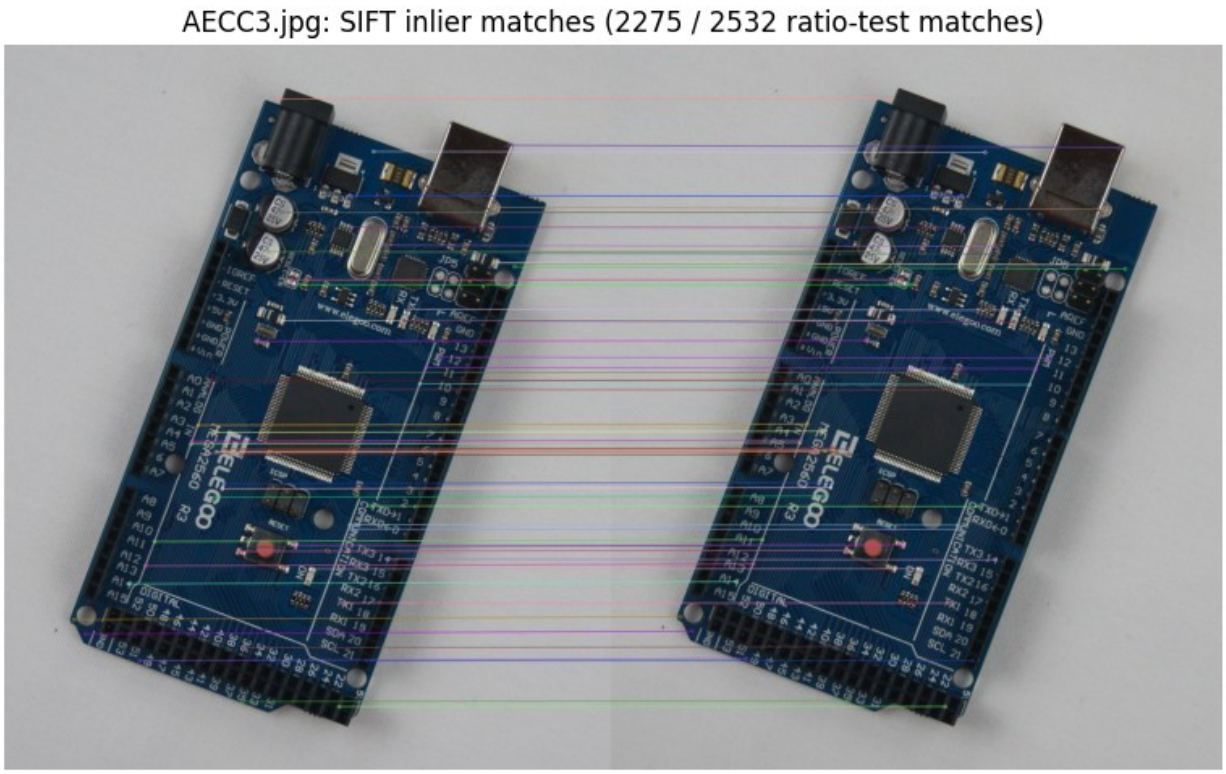
```
else:
    print("Successfully processed all five test PCB images.")
```

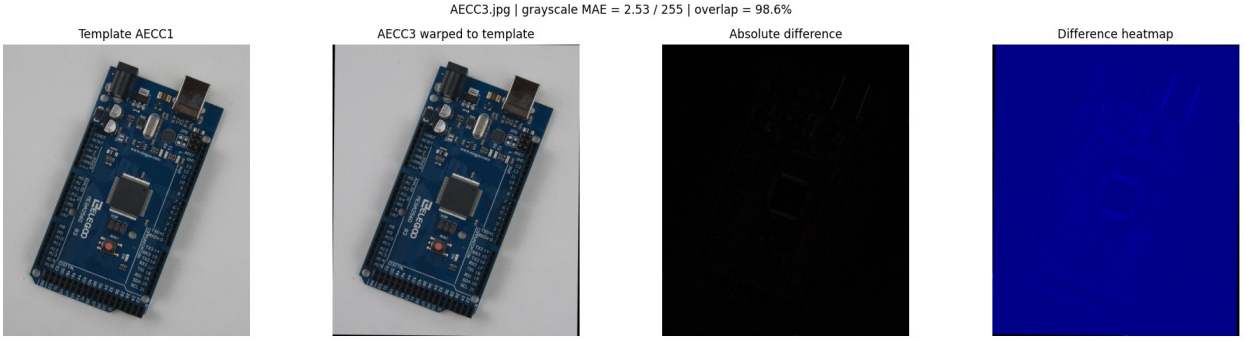
```
Processing AECC2.jpg ...
```



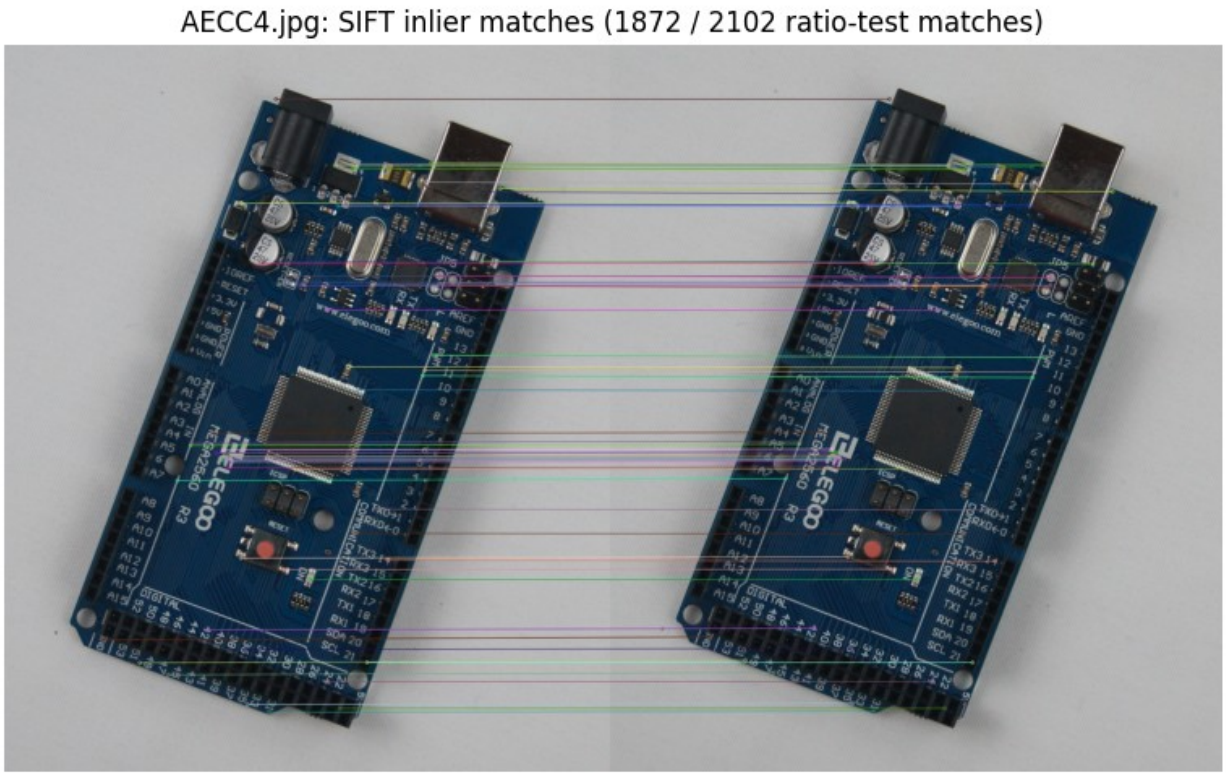


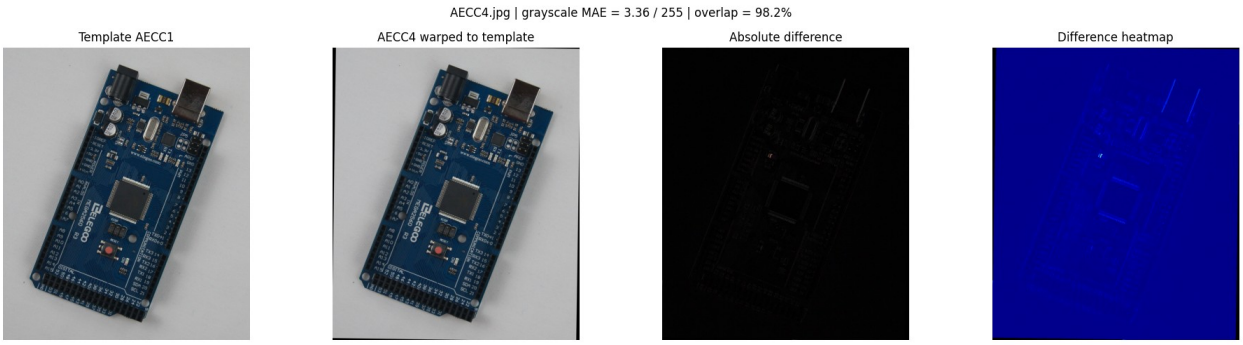

```
Processing AECC3.jpg ...
```



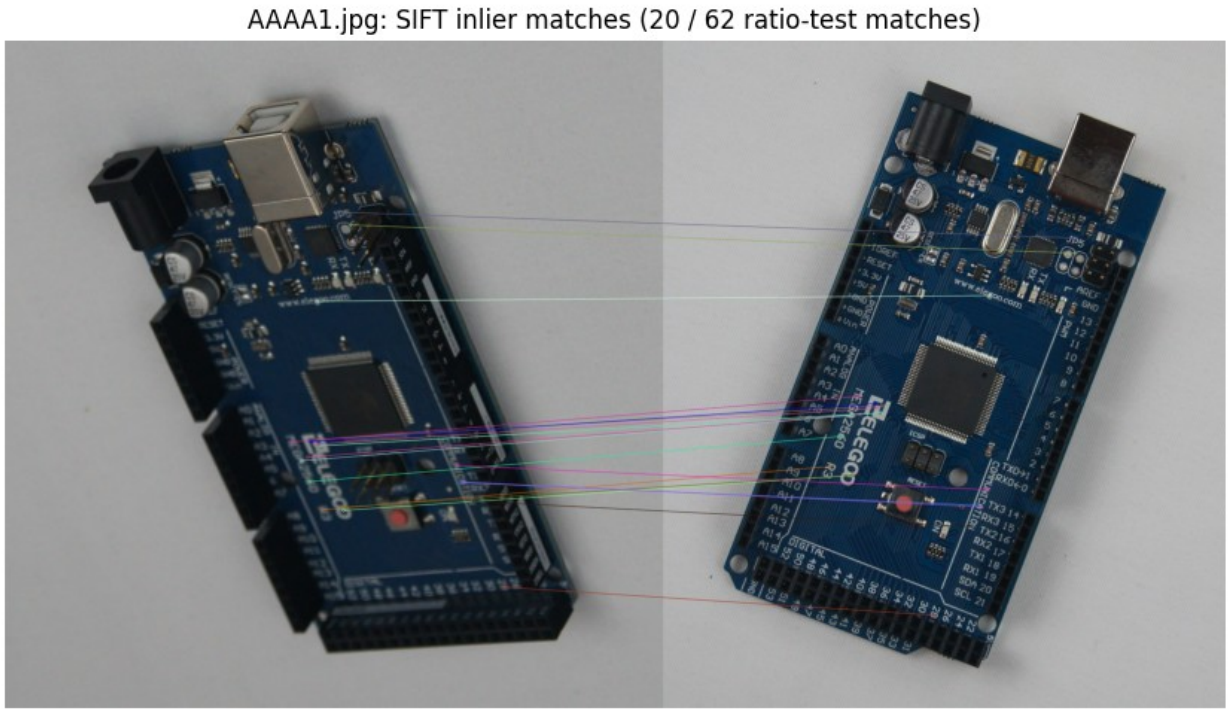


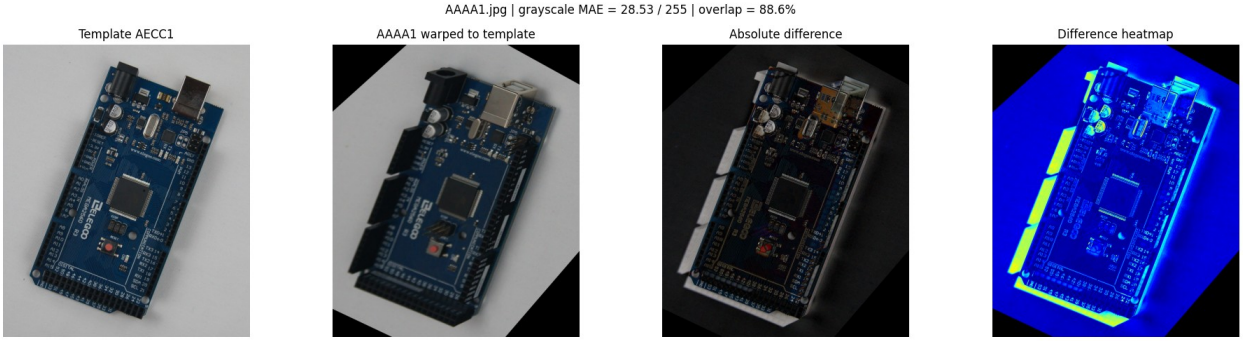

```
Processing AECC4.jpg ...
```



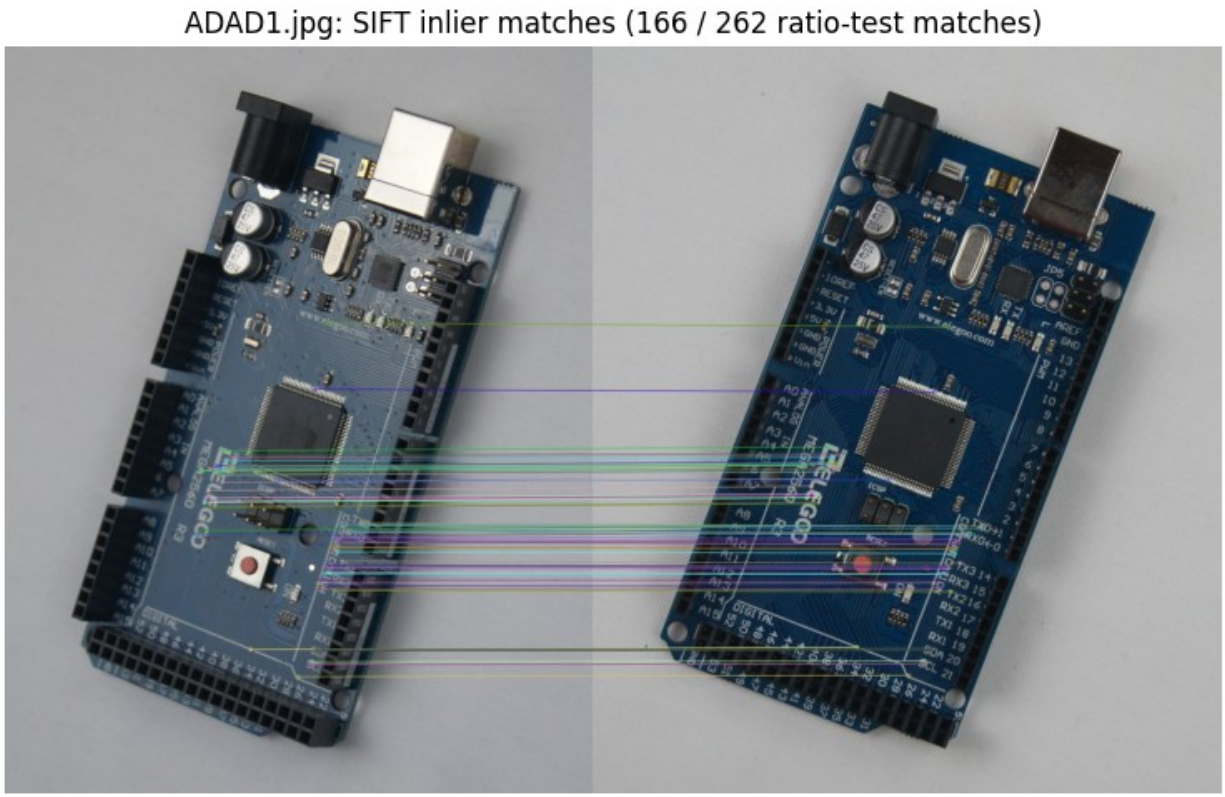


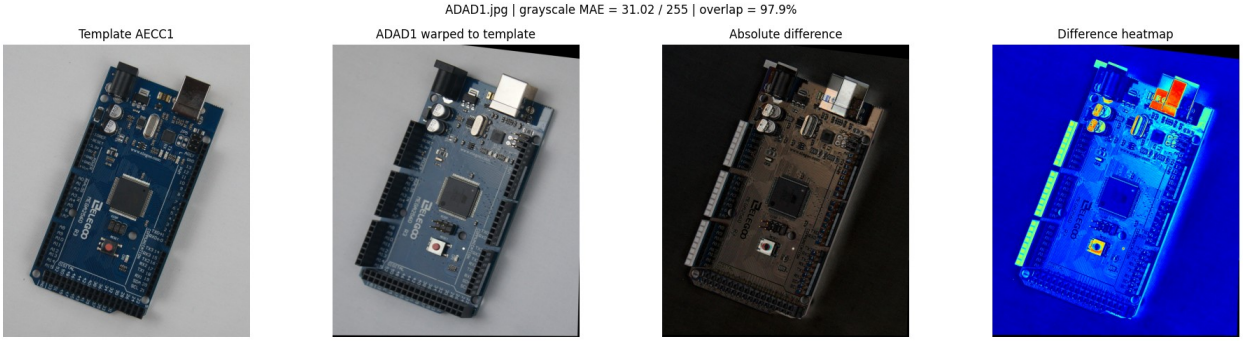

```
Processing AAAA1.jpg ...
```






```
Processing ADAD1.jpg ...
```





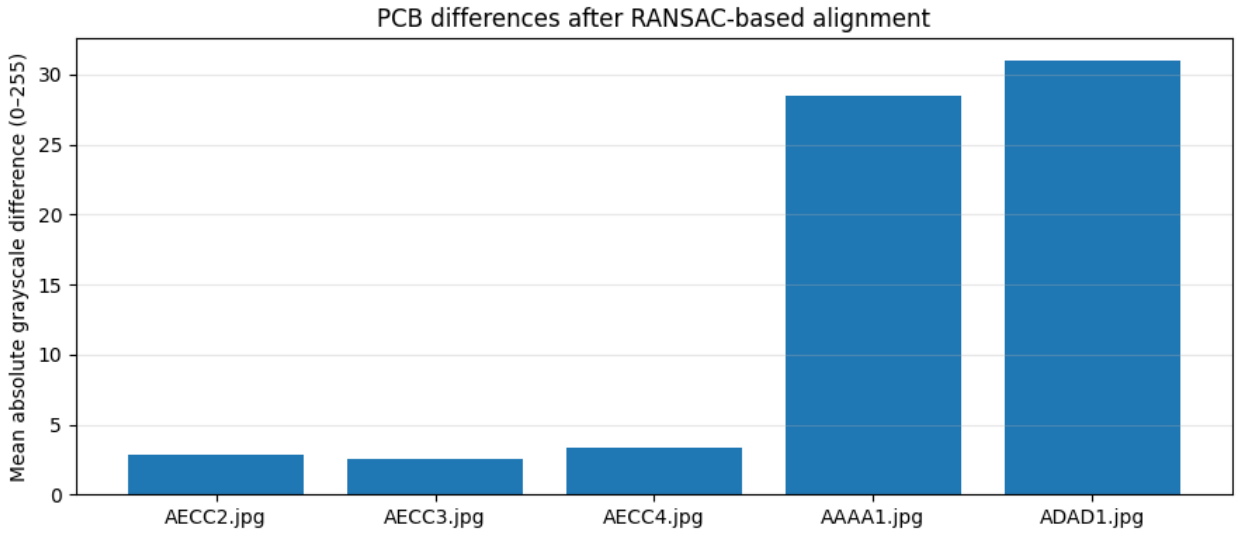

```
Successfully processed all five test PCB images.

metrics_df = pd.DataFrame(metrics)
if metrics_df.empty:
    raise RuntimeError("No successful alignments. Check the images and
matching parameters.")
display(metrics_df.round(3))
metrics_df.to_csv(RESULT_DIR / "q4_alignment_metrics.csv",
index=False)

fig, ax = plt.subplots(figsize=(9, 4))
ax.bar(metrics_df["image"], metrics_df["gray_MAE_0_to_255"])
ax.set_ylabel("Mean absolute grayscale difference (0–255)")
ax.set_title("PCB differences after RANSAC-based alignment")
```

```
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig(RESULT_DIR / "q4_difference_summary.png", dpi=150,
bbox_inches="tight")
plt.show()
```


```
{"summary":"{\n  \"name\": \"plt\",\n  \"rows\": 5,\n  \"fields\": [\n
{\n      \"column\": \"image\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 5,\n
\"samples\": [\n          \"AECC3.jpg\",\n          \"ADAD1.jpg\",\n
\"AECC4.jpg\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"ratio_test_matches\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 1160,\n        \"min\": 62,\n
\"max\": 2532,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          2532,\n          262,\n          2102\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"ransac_inliers\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1065,\n        \"min\": 20,\n        \"max\": 2275,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          2275,\n
166,\n          1872\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"inlier_ratio\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.25227504831037095,\n        \"min\":
0.323,\n        \"max\": 0.898,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          0.898,\n          0.634,\n          0.891\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"overlap_fraction\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.04300813876465708,\n        \"min\": 0.886,\n        \"max\":
0.986,\n        \"num_unique_values\": 5,\n        \"samples\": [\n
0.986,\n          0.979,\n          0.982\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"color_MAE_0_to_255\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
14.584693579228874,\n        \"min\": 2.656,\n        \"max\":
30.432,\n        \"num_unique_values\": 5,\n        \"samples\": [\n
2.656,\n          30.432,\n          3.498\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"gray_MAE_0_to_255\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
14.751160659419313,\n        \"min\": 2.53,\n        \"max\": 31.023,\
n        \"num_unique_values\": 5,\n        \"samples\": [\n
2.53,\n          31.023,\n          3.364\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```



Quantitative interpretation of the five PCB comparisons

The table and bar chart above are generated from the actual current run. The following cell ranks the five test images by valid-overlap grayscale MAE, so the discussion does not depend on hard-coded results from a previous execution. A low MAE indicates greater *pixel similarity after registration*, not proof that a board is defect-free. High MAE may reflect a different PCB variant, changes in components, illumination, viewpoint, or imperfect registration. Inlier count should be interpreted jointly with the **inlier ratio** and **overlap fraction**, rather than treated as an absolute quality score.

```
ranked =
metrics_df.sort_values("gray_MAE_0_to_255").reset_index(drop=True)
print("PCB comparisons ranked from smallest to largest grayscale
residual:")
display(ranked[["image", "ransac_inliers", "inlier_ratio",
"overlap_fraction", "gray_MAE_0_to_255"]].round(3))
if len(ranked) >= 2:
    best, worst = ranked.iloc[0], ranked.iloc[-1]
    print(f"Lowest residual: {best['image']}
({best['gray_MAE_0_to_255']:.2f}/255)")
    print(f"Highest residual: {worst['image']}
({worst['gray_MAE_0_to_255']:.2f}/255)")
    print("Interpret these alongside the match visualizations and
component-region heatmaps.")
```

```
PCB comparisons ranked from smallest to largest grayscale residual:
```


```
{"summary":"{\n  \"name\": \"    print(\\\"Interpret these alongside
the match visualizations and component-region heatmaps\",\n  \"rows\":
5,\n  \"fields\": [\n    {\n      \"column\": \"image\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n        \"samples\": [\n
\"AECC2.jpg\",\n          \"ADAD1.jpg\",\n          \"AECC4.jpg\"\n
```

```
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"ransac_inliers\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1065,\n        \"min\": 20,\n        \"max\": 2275,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          1895,\n
166,\n          1872\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"inlier_ratio\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.25227504831037095,\n        \"min\":
0.323,\n        \"max\": 0.898,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          0.89,\n          0.634,\n          0.891\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"overlap_fraction\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.04300813876465708,\n        \"min\": 0.886,\n        \"max\":
0.986,\n        \"num_unique_values\": 5,\n        \"samples\": [\n
0.981,\n          0.979,\n          0.982\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"gray_MAE_0_to_255\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
14.751160659419313,\n        \"min\": 2.53,\n        \"max\": 31.023,\
n        \"num_unique_values\": 5,\n        \"samples\": [\n
2.804,\n          31.023,\n          3.364\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```


```
Lowest residual: AECC3.jpg (2.53/255)
Highest residual: ADAD1.jpg (31.02/255)
Interpret these alongside the match visualizations and component-
region heatmaps.
```

4(e) Discussion — why are some component regions different?

USB Type-B connector, capacitors, and reset switch: These are raised three-dimensional components rather than points on a single flat PCB plane. A single homography can align the approximately planar board surface but cannot simultaneously correct the depth-dependent apparent displacement of tall components (parallax). Their sides, shadows, highlights, and partial occlusions can therefore produce strong residuals even if the component is physically unchanged. A component may also genuinely differ in shape, placement, or presence across PCB variants.

Photometric differences: Specular metallic connectors, shiny capacitor surfaces, changing light direction, camera exposure, and white balance can increase raw pixel differences. Pixel subtraction is not invariant to illumination.

Residual registration errors: Small reprojection errors are amplified near sharp edges, text, solder pads, and component boundaries. Lens distortion and the fact that the board is not perfectly planar can prevent perfect pixel-level alignment.

Interpretation of the measured results: The ranked table immediately above identifies which image has the smallest and largest residual in this execution. Differences between AECC2–AECC4 and AAAA1/ADAD1, if observed, can be consistent with changes in viewpoint or board appearance; the heatmaps and registered photographs are needed to establish their causes. The RANSAC inlier count and ratio indicate how many SIFT correspondences support a consistent mapping; they are *not* direct measures of PCB correctness. The MAE measures photometric mismatch over valid overlap, not solely manufacturing defects. Examine the saved alignment and difference maps together before concluding that a high-residual region represents a missing or damaged component.

Possible improvements: Use a board-only ROI, local illumination normalization, geometric alignment checks, component-level masks, and (when available) camera calibration. A planar homography is appropriate for the board surface, but not for all raised components.

Limitations: This experiment uses SIFT + RANSAC for global registration and absolute image differences for visualization. It is not a calibrated industrial defect detector; its outputs depend on viewpoint, lighting, image resolution, and overlap.